



BASE DE DATOS II

INFORME DEL PROYECTO 1

Grupo 3

Integrantes

De la Puente Rizo Patrón, Rodrigo (202410045)
Gurbillón Santa Cruz, Jimena Adela (202210159)
López Rojas, Yaritza Milagros (202410328)
Ordinola Ortega, Carlos David (202310448)
Walde Verano, Matías Sebastian (202410626)

Profesor

Sanchez Enriquez, Heider Ysaías

2026-10-04

Contenidos

1. Componentes del DBMS	(1)
1.a. Parser	(1)
1.a.i. Gramática	(1)
1.a.ii. Extensiones respecto de SQL estándar	(2)
1.a.iii. Validaciones del parser	(3)
1.a.iv. Errores	(3)
1.b. Executor	(3)
1.b.i. Modelo de ejecución	(3)
1.c. Buffer manager	(4)
1.d. File manager	(4)
1.e. Transaction manager	(5)
2. Índices y algoritmos incluidos	(6)
2.a. Archivos	(6)
2.a.i. Heap file	(6)
2.a.ii. Sequential file	(7)
2.b. Índices	(9)
2.b.i. Clustered B+	(9)
2.b.ii. Unclustered B+	(10)
2.b.iii. Extendible hashing	(11)
2.b.iv. Bitmap index scan	(12)
2.c. Algoritmos externos	(13)
2.c.i. External sorting (k -way merge)	(13)
2.c.ii. External hashing	(14)
2.d. Algoritmos para consultas espaciales	(15)
2.d.i. k -NN (k nearest neighbors)	(15)
2.d.ii. Búsquedas por rango	(16)
2.d.iii. Intersección con polígonos	(17)
3. Transacciones, concurrencia y manejo de conflictos	(18)
3.a. Transacciones	(18)
3.b. Atomicidad y UNDO	(18)
3.c. UNDO físico	(19)
3.d. REDO y recuperación	(19)
3.e. Control de concurrencia	(20)
3.f. Integración con el executor	(21)
4. Frontend	(21)
4.a. Estructura general y funcionamiento	(21)
4.b. Integración con Leaflet	(22)
5. Arquitectura final del DBMS	(23)
6. Benchmarking, comparaciones y testing	(23)
6.a. Heap file vs sequential file	(23)
6.a.i. Tiempo de inserción	(24)
6.a.ii. Tiempo de búsqueda por clave primaria	(25)

6.a.iii. Espacio en disco utilizado	(28)
6.a.iv. Tiempo de reorganización	(29)
6.b. B+ clustered vs B+ unclustered vs extendible hashing	(30)
6.b.i. Búsqueda por igualdad exacta	(30)
6.b.ii. Búsqueda por rango	(30)
6.b.iii. Ordenamiento	(31)
6.c. Búsqueda secuencial vs R-tree vs PostGIS	(31)
6.c.i. Consultas por rango	(35)
6.c.ii. Consultas de k -NN	(37)
6.c.iii. Discusión y limitaciones	(38)

1. Componentes del DBMS

Para examinar los detalles completos de la implementación, la arquitectura de clases y las rutinas de bajo nivel descritas en este informe, se recomienda consultar el código fuente. Puede revisar el repositorio oficial del proyecto en el siguiente enlace:

Repositorio del Proyecto (GitHub)

1.a. Parser

El parser recibe una cadena SQL y devuelve un árbol de sintaxis abstracta que el executor recorre para resolver la consulta. Se implementó un subconjunto propio de SQL, suficiente para ejercitar todas las estructuras del motor: las dos organizaciones de archivo, los cuatro tipos de índice, las operaciones de consulta y modificación, las transacciones y las consultas espaciales.

El parser no valida contra el catálogo. No sabe qué tablas existen ni qué tipo tiene cada columna, así que las comprobaciones que dependen de esa información que una columna exista, que los tipos coincida quedan en el executor.

1.a.i. Gramática

{X}* indica cero o más repeticiones, [X] un elemento opcional y | separa alternativas. Los terminales van en mayúsculas; id, num y str son identificadores, números y cadenas.

```
Program      ::= Stmt {; Stmt}* [;]

Stmt         ::= CreateTable | CreateIndex | DropTable | Select
               | Insert | Delete | Update | Transaction | Explain

CreateTable  ::= CREATE TABLE id ( ColDec {, ColDec}* ) USING FileOrg
ColDec       ::= id Type [PRIMARY KEY]
Type         ::= INT | FLOAT | BOOL | DATE | POINT | VARCHAR ( num )
FileOrg      ::= HEAP | SEQUENTIAL

CreateIndex  ::= CREATE INDEX ON id ( id ) USING IndexKind [CLUSTERED]
IndexKind    ::= BTREE | HASH | BITMAP | RTREE

DropTable    ::= DROP TABLE id

Select       ::= SELECT SelList FROM id [Join] [Where] [GroupBy]
               [OrderBy] [Limit]
SelList      ::= * | Proj {, Proj}*
Proj         ::= ColRef | AggFun ( ColRef | * )
AggFun       ::= COUNT | SUM | AVG | MIN | MAX
Join         ::= JOIN id ON ColRef = ColRef
Where        ::= WHERE Cond
GroupBy      ::= GROUP BY ColRef
OrderBy      ::= ORDER BY OrderExpr [ASC | DESC]
Limit        ::= LIMIT num

Insert       ::= INSERT INTO id VALUES ( Value {, Value}* )
Delete       ::= DELETE FROM id WHERE Cond
Update       ::= UPDATE id SET Asig {, Asig}* WHERE Cond
```

```

Asig          ::= id = Value

Transaction   ::= BEGIN TRANSACTION | END TRANSACTION | ROLLBACK
Explain       ::= EXPLAIN [ANALYZE] Stmt

Cond          ::= AndCond {OR AndCond}*
AndCond       ::= Pred {AND Pred}*
Pred          ::= Operando RelOp Value
               | ColRef BETWEEN Value AND Value
               | DentroDe
               | ( Cond )
Operando      ::= ColRef | Distancia
RelOp         ::= = | != | <> | < | <= | > | >=

Distancia     ::= FunDist ( OpEspacial , OpEspacial )
FunDist       ::= DISTANCIA | DISTANCIA_EUCLIDIANA | DISTANCIA_GEODESICA
DentroDe      ::= DENTRO_DE ( ColRef , Polygon )
Polygon       ::= POLYGON ( Point {, Point}* )
Point         ::= POINT ( Numero , Numero )
OpEspacial    ::= Point | ColRef
OrderExpr     ::= Distancia | ColRef

ColRef        ::= id | id . id
Numero        ::= [-] num
Value         ::= [-] num | str | TRUE | FALSE | Point

```

1.a.ii. Extensiones respecto de SQL estándar

La cláusula `USING` en `CREATE TABLE` y `CREATE INDEX` permite elegir explícitamente la organización del archivo y el tipo de índice. En un gestor de producción esa decisión la toma el optimizador y no se expone al usuario; acá hace falta porque la comparación experimental exige forzar cada técnica por separado y medirla.

El tipo `POINT` almacena un par de coordenadas. Un literal se escribe `POINT(longitud, latitud)`, con el orden de PostGIS: primero la coordenada X. Lima es `POINT(-77.0428, -12.0464)`.

Sobre columnas `POINT` se definen tres funciones:

Función	Qué calcula	Unidad
<code>distancia</code>	distancia euclidiana	grados
<code>distancia_geodesica</code>	distancia Haversine	metros
<code>dentro_de</code>	pertenencia a un polígono	booleano

`distancia_euclidiana` es un alias de `distancia`. Las dos primeras devuelven un número, así que se combinan con un operador de comparación y pueden usarse como criterio de `ORDER BY`; `dentro_de` ya es un predicado completo.

1.a.iii. Validaciones del parser

El parser rechaza algunas combinaciones que son sintácticamente válidas pero no tienen sentido para el motor:

- Un índice **HASH** no puede ser **CLUSTERED**: el hash no preserva el orden de las claves, así que no puede definir el orden físico de los registros.
- Un índice **BITMAP** tampoco: guarda máscaras de bits, no registros ordenados.
- Solo **COUNT** admite ***** como argumento.
- Un **POLYGON** necesita al menos tres vértices.

1.a.iv. Errores

Se distinguen errores léxicos, sintácticos y semánticos, y todos reportan la línea y la columna donde ocurren:

```
SELECT id cliente FROM t;
```

Error sintáctico en línea 1, columna 11: se esperaba FROM

```
CREATE INDEX ON t (c) USING HASH CLUSTERED;
```

Error semántico en línea 1, columna 43: un índice HASH no puede ser CLUSTERED

La posición se guarda como dato separado del mensaje, lo que permite que la interfaz web marque el error directamente sobre el editor de consultas.

1.b. Executor

El executor recorre el AST que produjo el parser y resuelve la consulta contra el motor de almacenamiento. Es la única capa que conoce las dos cosas: traduce los tipos del parser a los del catálogo, los nombres de columna a posiciones dentro del registro, y decide con qué índice resolver cada predicado.

1.b.i. Modelo de ejecución

Se implementó el modelo de iteradores de Volcano: cada operador del plan expone la misma interfaz y pide registros al operador que tiene debajo, de a uno por vez. El consumo es por demanda, así que ningún paso materializa su resultado completo para pasárselo al siguiente.

La ventaja concreta es que un **LIMIT** corta la ejecución aguas arriba. Sobre una tabla de 3.000 filas:

Consulta	Filas leídas por el scan
SELECT id FROM t LIMIT 5;	5
SELECT id FROM t;	3.000

El acceso se detiene apenas el **LIMIT** quedó satisfecho; no se leen las 2.995 filas restantes. El mismo mecanismo evita cargar en memoria el resultado intermedio de un **WHERE** poco selectivo.

1.c. Buffer manager

Implementada bajo el patrón Singleton para garantizar una única instancia compartida en todo el sistema, la clase **BufferManager** actúa como el intermediario fundamental entre las capas superiores del motor de base de datos y el almacenamiento físico. Su responsabilidad principal es minimizar las costosas operaciones de entrada/salida (I/O) en disco, administrando un caché global en memoria compuesto por contenedores lógicos denominados **Frame**. Estos marcos retienen temporalmente las páginas y controlan su ciclo de vida mediante un mecanismo de fijación (**pin_count**), el cual garantiza que las páginas en uso no sean sobrescritas mientras se operan en memoria.

Para gestionar eficientemente este espacio limitado, el componente utiliza el algoritmo de reemplazo del Reloj (**Clock algorithm**). Cuando una capa superior solicita una página mediante **fetch_page** y el búfer está lleno, el gestor evalúa el historial de accesos de cada marco para seleccionar y desalojar una «víctima» inactiva, dando prioridad de permanencia a las páginas referenciadas recientemente para optimizar el rendimiento general.

Además, el gestor es una pieza crítica para la consistencia transaccional de la base de datos. Rastrea de forma continua las páginas que han sido alteradas (**dirty**) y se sincroniza estrechamente con la infraestructura WAL al capturar los cambios en memoria mediante el método **mark_dirty**. Finalmente, sus rutinas de vaciado (**flush_page**, **flush_all**) garantizan que todas las modificaciones en caché se persistan de manera segura a través del **FileManager** antes de liberar memoria o apagar el motor.

1.d. File manager

La clase **FileManager** actúa como el componente de más bajo nivel en el motor de almacenamiento de la base de datos. Su responsabilidad principal es abstraer las operaciones del sistema operativo sobre el archivo físico, permitiendo a las capas superiores leer y escribir información en páginas. Además, integra la infraestructura base para el registro de transacciones (*Write-Ahead Logging* o WAL).

A nivel lógico, el archivo se estructura en tres secciones principales: una Cabecera inicial de **file_header_size** bytes destinada a los metadatos globales, una Página 0 (Overflow) reservada para los registros que superan el tamaño estándar, y un conjunto de Páginas regulares (de la 1 en adelante) de tamaño fijo (**page_size**) donde se almacenan los datos normales. Dentro de este esquema, la ubicación exacta de cada registro se identifica de forma única mediante su Record ID (**rid**), calculando su posición con la fórmula $rid = phys_page_id \times records_per_page + slot_id$.

El constructor de **FileManager** gestiona la apertura o creación del archivo en modo binario y establece un sistema de acceso de tamaño fijo que garantiza la correcta alineación en disco. Sus métodos de lectura y escritura operan calculando offsets exactos, ya sea de forma exclusiva para la cabecera (**read_header** / **write_header**) o para los bloques de datos (**read_page** / **write_page**). Adicionalmente, el gestor permite expandir el almacenamiento de forma dinámica mediante **allocate_page**, que se posiciona al final del archivo, añade un bloque de bytes nulos y devuelve el índice de la nueva página.

Para garantizar las propiedades ACID y la tolerancia a fallos, la clase integra el protocolo de registro de transacciones (WAL) a través de la inyección estática de dependencias con `configure_wal`. Esto permite que cualquier modificación estructural se registre de manera transparente mediante `_log_physical()`. Finalmente, el componente asegura la persistencia de los datos ofreciendo el método `flush()` para vaciar los buffers hacia el sistema operativo, y `force()` (basado en `os.fsync()`), un paso indispensable para obligar la escritura física en el hardware antes de confirmar cualquier transacción.

1.e. Transaction manager

El `TransactionManager` es el componente encargado de manejar el ciclo de vida de las transacciones y su relación con el WAL. Mantiene en memoria una estructura por cada transacción conocida, con su identificador, estado actual y último LSN registrado.

Los estados utilizados son:

- **ACTIVE**: la transacción acepta nuevas operaciones y puede ejecutar **COMMIT** o **ROLLBACK**.
- **COMMITTED**: la transacción terminó correctamente y su commit ya fue forzado al WAL.
- **ABORTED**: la transacción fue cancelada y sus cambios registrados fueron deshechos.

Cuando se ejecuta **BEGIN TRANSACTION**, el manager genera un identificador nuevo, escribe un registro **BEGIN** y fuerza el WAL. Cada mutación posterior se registra mediante `log_update()`. Este método crea un registro de tipo **UPDATE**, conserva el `prev_lsn` de la transacción y actualiza su `last_lsn`. La confirmación se realiza mediante `commit()`. Primero se agrega el registro **COMMIT** utilizando el último LSN de la transacción como `prev_lsn`. Luego se ejecuta `force()` sobre el WAL. Solo después de que `force()` termina correctamente se cambia el estado a **COMMITTED**.

El rollback se realiza mediante `rollback()`. El manager busca los registros de la transacción, escribe un registro **ABORT** y los procesa en orden inverso. Para cada registro llama a un `undo_handler`, que conoce cómo restaurar la fila, página, header o metadata correspondiente. Después de aplicar el UNDO, se agrega un CLR, que deja constancia en el WAL de la compensación realizada. Finalmente, el WAL se fuerza y la transacción pasa al estado **ABORTED**.

El `TransactionManager` no conoce los formatos de las tablas ni la estructura interna de las páginas. Su responsabilidad es coordinar estados, LSN y eventos del WAL; la aplicación concreta del UNDO y redo pertenece al `executor`, al `StorageManager` y al `RecoveryManager`. Esta separación permite que el manager controle la durabilidad sin depender de una organización física específica, como `HeapFile` o `SequentialFile`.

El componente también reconstruye el estado de las transacciones cuando se vuelve a abrir el WAL. Si encuentra un **BEGIN** sin un **COMMIT** o **ABORT** posterior, la transacción queda como **ACTIVE** y `RecoveryManager` la trata como incompleta, aplicando UNDO durante el arranque.

2. Índices y algoritmos incluidos

2.a. Archivos

2.a.i. Heap file

El heap file es la estructura física principal para el almacenamiento de tuplas en el DBMS, la cual permite gestionar los registros de manera paginada sin imponer un orden estricto sobre ellas. El archivo está compuesto por una secuencia de páginas guardadas secuencialmente en el disco, cada una de 4096 bytes. Hay dos tipos de páginas: *directory pages* y *data pages*.

Las directory pages mantienen la cantidad de espacio libre por página. Sus cabeceras ocupan 8 bytes, pues mantienen la cantidad de páginas que este directory page almacena (`page_count`) y el índice de la siguiente página de directorio (`next_dir_page_id`). Las entradas dentro de la página misma son de 2 bytes (un `unsigned short`) por lo que, en total, cada directory page contiene información del tamaño de las siguientes 2044 páginas. La primera directory page es la página 0 y, una vez se llena, inmediatamente se crea una nueva para las siguientes 2044 páginas en el índice 2045, y así sucesivamente.

Por otra parte, las data pages funcionan como una slotted page estándar (sin importar que los registros son de tamaño fijo o no), por lo que tienen slots que se escriben de forma descendente (fin del header hacia abajo), y registros serializados que se escriben de forma ascendente (fin de la página hacia arriba). Contiene 12 bytes de cabecera: el puntero al inicio del área de registros (`offset`), la cantidad de registros en la página (`size`) y el puntero al inicio del espacio libre en el área de registros (`first_free_slot`). Cada slot ocupa 8 bytes para almacenar un puntero al inicio del registro (`record_offset`) y su tamaño (`record_size`). Entre el área de slots y el área de registros, hay una región no asignada ubicada, considerada espacio libre.

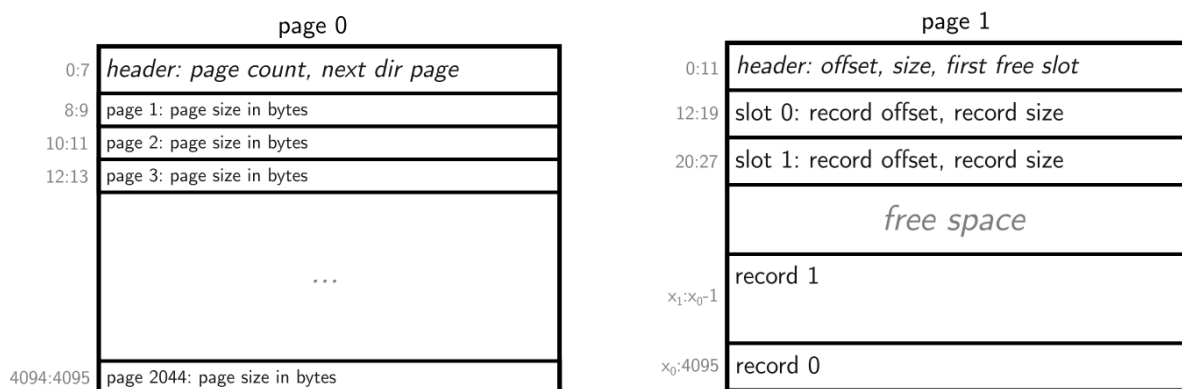


Figura 1: Ejemplo de directory page (izquierda) y data page (derecha) en el heap file.
Elaboración propia.

Para una búsqueda, no hay más remedio que iterar a través de todas las páginas de datos hasta encontrar los registros deseados, por lo que es $\mathcal{O}(n)$.

Para una eliminación que evite cambiar los RIDs de los registros restantes, se aplica una estrategia de free list para reutilizar slots eliminados. Se accede a la página de datos y al

slot correspondiente, cuyo `record_size` se establece en 0 y cuyo `record_offset` se utiliza para almacenar el siguiente slot libre. El slot no se elimina, ya que hacerlo modificaría los IDs de los slots posteriores. Finalmente, se actualiza en el directory page la cantidad de espacio libre disponible de la página.

Para insertar, se busca evitar recorrer todas las páginas de datos en $\mathcal{O}(n)$. Para ello, se recorren las directory pages y sus entradas hasta encontrar una página de datos con espacio suficiente para el nuevo registro. Una vez encontrada, se accede a ella y se inserta el slot y el registro serializado. Si no se encuentra espacio suficiente en ninguna directory page, se crea una nueva data page. Esto tiene complejidad $\mathcal{O}(m)$ (donde m es la cantidad de páginas de directorio), lo cual es mucho mejor dado que $m \ll n$. Si es que hay espacio para el registro pero no de manera contigua, entonces se compactan los registros de la página con `defragment`. Dado que sólo se aplica a una página en RAM, se mantiene la complejidad de $\mathcal{O}(m)$.

En términos de memoria, dado que estamos forzando un slotted page sin importar que los registros sean de longitud fija o variable, entonces estamos usando más memoria de la necesaria para registros de longitud fija. Esto causa que las páginas se llenen más rápido, pero favorece la simplicidad del código al tener sólo una implementación.

2.a.ii. Sequential file

El archivo secuencial almacena registros ordenados según una clave definida por `key_index` dentro del esquema de la tabla. El orden lógico se mantiene mediante una lista enlazada simple: cada registro almacena el RID de su sucesor (`next_rid`). De este modo, la secuencia completa puede reconstruirse recorriendo los registros desde `first_rid`. Los RIDs se mantienen durante las inserciones y eliminaciones normales, pero pueden cambiar después de una reorganización.

Este archivo tiene un file header de 32 bytes, que consiste en 8 enteros:

- la cantidad de páginas de datos (`n_pages`),
- el RID del primer registro en la secuencia lógica, almacenados como dos componentes separados (`page_id`, `slot_id`), donde $(-1, -1)$ representa una secuencia vacía.
- la cantidad de registros activos (`n_records`),
- la cantidad de registros eliminados (`n_deleted`),
- la cantidad de páginas de overflow (`n_overflow_pages`), y
- la cantidad de registros en páginas de overflow (`n_overflow_records`).
- el índice físico de la última página de overflow utilizada (`overflow_tail_page`), por default 0 cuando o existe ninguna.

Cada registro lógico contiene sus atributos `params`, además del RID de su sucesor y una flag de un byte `deleted` para facilitar la eliminación. Hay dos tipos de páginas: *overflow pages* y *data pages*, ambas conteniendo registros serializados. El hecho que haya dos secciones para registros hace que las operaciones de búsqueda, inserción y eliminación sean más complejas de implementar que en el heap file.

Las páginas de overflow son no ordenadas a propósito, para que la inserción sea lo más rápida posible (prácticamente como en un heap file, escribiendo siempre en la última

página de overflow, y reservando una nueva al final del archivo sólo cuando la actual se llena). No es necesario recorrerlas por índice físico para encontrar un registro, ya que todas ellas están enlazadas dentro de la misma secuencia lógica.

Las páginas de datos contienen los datos ordenados por llave, pero su implementación depende de si los registros son de tamaño fijo o variable (esta última funcionando como el slotted page visto en el heap file). La diferencia común a ambas es que la eliminación no es física sino lógica, a través de un byte `deleted` al final de todos los registros serializados, justo después del `next_id`, evitando reorganización inmediata.

- En el caso de longitud variable, la página tiene una cabecera de 12 bytes: el puntero al inicio del área de registros (`n_overflow_pages`), la cantidad de registros (`n_overflow_records`) y el puntero al primer slot libre (`overflow_tail_page`). Los slots son de 8 bytes, igual que el slotted page de heap file exceptuando al free list al siguiente eliminado (pues no es necesario aquí, los slots eliminados no se reciclan, se posterga hasta el siguiente `reorganize`).
- En el caso de longitud fija, la página es un arreglo de slots de tamaño constante organizados de forma ascendente desde el inicio de la cabecera de 4 bytes que guarda la cantidad de registros, con lo que una página de 4096 bytes almacena como máximo $\frac{4096-4}{\text{slot_size}}$ registros.

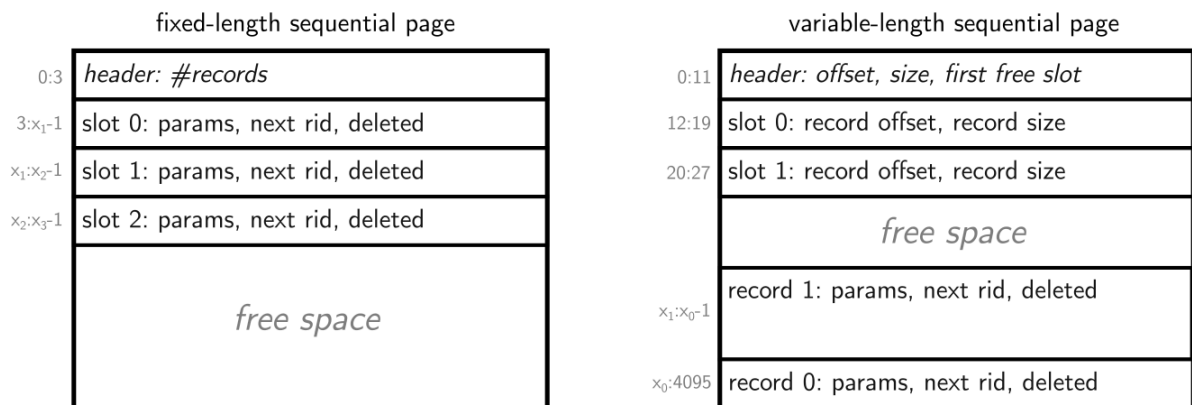


Figura 2: Ejemplo de una página de longitud fija (izquierda) y variable (derecha).
Elaboración propia.

La operación `search(key)` aprovecha el orden de las páginas principales para localizar una zona cercana a la clave buscada. Primero realiza una búsqueda binaria entre las páginas principales, comparando la clave del primer registro de cada página. Luego recorre la secuencia lógica mediante los enlaces `next_rid`, incluyendo registros ubicados en páginas de overflow, hasta encontrar registros con una clave mayor a la buscada. Finalmente, retorna únicamente los registros activos cuya clave coincide.

La búsqueda tiene una complejidad aproximada de $\mathcal{O}(\lg n + m)$, donde n es el número de páginas principales y m representa el recorrido adicional producido por los registros almacenados en overflow.

La eliminación es lógica, pues se emplea la flag de `deleted`. Los datos del registro siguen existiendo en el lugar en el que se encuentre, sólo que la búsqueda y la inserción ignoran

los datos marcados como eliminados. Involucra primero buscar el registro en cuestión (como fue descrito en la búsqueda), y marcar su `deleted` como verdadero. Se actualizan `n_deleted` y `n_records`, y se puede ejecutar una reorganización si se cumple el load factor del 30% de registros eliminados ($\frac{\text{eliminados}}{\text{totales}} \geq 0.3$) o, en el caso de `delete_by_key`, también puede producirse una reorganización si alguna página principal afectada queda sin registros activos. Sin reorganización, la eliminación tiene una complejidad aproximada de $\mathcal{O}(\lg n + m)$.

La inserción es de tipo lazy: los nuevos registros no se incorporan inmediatamente a las páginas principales ordenadas, sino que se escriben en el área de overflow. Primero se determinan el predecesor y sucesor lógicos mediante `_find_neighbors`. Luego, el nuevo registro se inserta en la última página de overflow disponible y se actualizan los enlaces `next_rid` correspondientes. Si la última página de overflow no tiene espacio suficiente, se crea una nueva página. Después de la inserción se actualizan los contadores del archivo. Cuando existen al menos 20 registros activos y la proporción de registros en overflow alcanza el 30%, se ejecuta una reorganización. Sin considerar una reorganización, la inserción tiene una complejidad aproximada de $\mathcal{O}(\lg n + m)$.

La reorganización (a través de `reorganize`) reconstruye el archivo para eliminar los registros borrados, compactar el espacio y devolver todos los registros activos a las páginas principales en orden ascendente por clave. Primero se recorre la secuencia lógica, se recopilan los registros activos y se ordenan (sea con el sort de Python, o external sort si los registros no caben en RAM). Una vez obtenido el orden final, los registros se escriben nuevamente en las páginas principales y se reconstruyen sus enlaces `next_rid`. El área de overflow se reinicia y el archivo se trunca para conservar únicamente las páginas principales necesarias. Si se proporcionó `track_rid`, la reorganización devuelve el nuevo RID correspondiente al registro original que se estaba siguiendo.

2.b. Índices

2.b.i. Clustered B+

La clase `BPlusTreeClustered` implementa un índice agrupado extendiendo la funcionalidad de un Árbol B+ genérico (`BPlusTreeBase`). A diferencia de un índice secundario tradicional, esta estructura está fuertemente acoplada a un archivo de datos subyacente (`SequentialFile`). Su propósito principal es coordinar el almacenamiento físico de los registros y mantener un árbol de búsqueda eficiente donde las hojas almacenan únicamente identificadores de registro (RID: página y `slot`), apuntando a los datos reales en el archivo secuencial.

En lugar de almacenar los atributos del registro directamente en los nodos del árbol, el índice delega el manejo físico de los datos al `SequentialFile`.

- **Inserción (`_store_record`):** Al recibir un nuevo registro, primero se inserta físicamente en el archivo secuencial. Este retorna su ubicación exacta (`page_id`, `slot_id`), la cual es encapsulada en un objeto RID y almacenada en la hoja correspondiente del Árbol B+.

- **Lectura y Eliminación:** Métodos como `_fetch_record` y `_delete_record` utilizan estas referencias directas para recuperar los datos binarios o marcar registros como eliminados lógicamente en el archivo base, asegurando una recuperación en tiempo constante ($O(1)$) una vez atravesado el árbol.

El aspecto más complejo de esta clase es su resiliencia ante cambios estructurales del archivo de datos. Dado que el `SequentialFile` puede someterse a rutinas de compactación o reorganización interna (lo que altera irremediabilmente las posiciones físicas o RID de los registros), el índice B+ quedaría apuntando a direcciones inválidas si no se actualizara.

Para solucionar esto de manera transparente, el sistema implementa un control de versiones semántico:

1. **Monitoreo de Estado:** El árbol guarda el estado estructural del archivo mediante la variable `_reorganize_count`.
2. **Detección de Desfase (`_check_reindex`):** Tras cualquier operación de escritura o borrado físico, el gestor compara su contador con el del `SequentialFile`. Si detecta una discrepancia, levanta la excepción de control interna `_ReindexNeeded`.
3. **Recuperación Silenciosa:** Los métodos públicos `insert` y `delete` actúan como envoltorios transaccionales. Al capturar la excepción `_ReindexNeeded`, suspenden la operación actual y disparan el método `_reindex()`.
4. **Reconstrucción (`_reindex`):** Este proceso reinicia la estructura del árbol (`_init_empty_index`) y realiza un escaneo secuencial (*Full Scan*) sobre el `SequentialFile`. Ignora los registros marcados como eliminados y reinserta secuencialmente en el nuevo Árbol B+ las claves junto con sus direcciones físicas actualizadas.

Finalmente, una vez que la reindexación concluye y el contador se sincroniza, la operación original que disparó la alarma se retoma o se da por completada, garantizando una consistencia absoluta de los datos sin requerir intervención externa ni exponer los fallos lógicos a las capas superiores.

2.b.ii. Unclustered B+

La clase `BPlusTreeUnclustered` implementa un índice secundario o no agrupado, extendiendo la lógica base del Árbol B+ (`BPlusTreeBase`). A diferencia del índice agrupado, esta estructura opera sobre un archivo de tipo montículo o **Heap** (`HeapFile`), lo que significa que el orden físico de los datos en el disco es arbitrario y no guarda relación con el orden de las claves en el índice. En consecuencia, las hojas del árbol actúan exclusivamente como un directorio de punteros lógicos (`ref`), delegando la persistencia, lectura y borrado de los registros completos al gestor del `HeapFile`.

La abstracción de almacenamiento se consolida mediante los métodos `_store_record`, `_fetch_record` y `_delete_record`. En este diseño, el árbol B+ ignora por completo la ubicación física exacta y la estructura interna del registro; simplemente pasa los parámetros al `HeapFile` y almacena la referencia (`ref`) devuelta por este último. Esto

permite tener múltiples índices no agrupados apuntando al mismo **HeapFile** sin duplicar la información base.

El desafío algorítmico principal de un índice secundario es que el atributo indexado no necesariamente es único (por ejemplo, un índice sobre la columna «país»). Para soportar múltiples registros compartiendo la misma clave, la clase implementa mecanismos especializados:

1. **Registro de Duplicidad (`_insert_ref`):** Durante la inserción, el sistema evalúa si la clave ya existe en el árbol mediante una búsqueda previa. Si es así, la registra en la estructura en memoria `_keys_with_duplicates`. Esto optimiza la respuesta posterior para saber dinámicamente si debe retornar un valor único o una lista de referencias.
2. **Búsqueda con Escaneo Secuencial (`search`):** Para garantizar que se recuperen todos los registros asociados a una clave duplicada, la búsqueda modifica el descenso tradicional del Árbol B+:
 - Primero, atraviesa los nodos internos forzando la ruta hacia la primera aparición de la clave en el extremo izquierdo (`find_leftmost_child`).
 - Una vez en la hoja correspondiente, inicia un escaneo lineal (**range scan**). Recorre las entradas y transita por las hojas enlazadas contiguas (`next_leaf_id`) acumulando coincidencias.
 - El escaneo se detiene (corte anticipado o **early exit**) en el momento en que se detecta una clave mayor estricta (`entry_key > key`), optimizando el rendimiento.
3. **Borrado Preciso (`delete_ref`):** En presencia de claves duplicadas, la eliminación no puede basarse únicamente en la clave de búsqueda, ya que esto podría destruir múltiples referencias válidas indiscriminadamente. Por ello, el método `delete_ref` requiere tanto la clave (`key`) como el puntero exacto (`ref`). El algoritmo desciende nuevamente a la hoja más a la izquierda y barre secuencialmente las entradas hasta encontrar la tupla exacta (`key, ref`). Una vez localizada, elimina la entrada, reescribe la página (`_rewrite`) e invoca explícitamente `_save_page` para asegurar la consistencia del bloque en disco.

2.b.iii. Extendible hashing

El índice de extendible hashing se usa para encontrar los RID de los registros cuyas tuplas son iguales a una `key` dada, los cuales se encuentran almacenados internamente en buckets. Esto se lleva a cabo mediante el hashing de la llave mediante MurmurHash3 de 32 bits, de los cuales se extrae los D bits menos significativos, donde D es el valor de profundidad global (`depth`) de la estructura.

El índice comienza con $2^{D_{\text{inicial}}}$ buckets. Estos se acceden mediante un directorio almacenado en páginas (`DirectoryPage` en la implementación) que siempre mantiene 2^D entradas diferentes. Estas apuntan al bucket en donde se encuentran los elementos cuyos bits menos significativos del hash de su llave coinciden con la entrada del diccionario. Cada bucket además mantiene un atributo `local_depth` que representa la cantidad de bits de su entrada, que siempre serán los menos significativos de las entradas que apunten a este. Eventualmente, cuando un bucket no tenga espacio para la inserción

de un nuevo registro, se divide en dos: si `local_depth` es menor que D , los dos buckets resultantes pasan a tener un bit más que difiere entre los dos, y todas las entradas que apuntan a ellos proceden a fijarse al nuevo bucket correspondiente. Si, por el otro lado, `local_depth` es igual a D , se debe duplicar el tamaño del directorio mediante `double_in_size`, que duplica el tamaño (y, si este ya llena al menos una página, duplica la cantidad de páginas que se usan). Una vez se alcance el factor de profundidad máximo ($D = 20$ por defecto para esta implementación, aunque puede tomar cualquier valor de 11 a 20), se comienzan a crear buckets de overflow que se encadenan a los anteriores mediante el atributo `next_bucket_page`. Al momento de la búsqueda, se verifica completamente el bucket de la entrada correspondiente y luego los buckets que este tenga encadenados.

Los registros se representan mediante la clase `KV`, que contiene una `key` (de tipo y longitud arbitrarios), el `rid` y un booleano `deleted` de si está eliminado. Estos se almacenan en `slotted pages` al poder ser las llaves de longitud variable (`VariableBucketPage` en la implementación). Cada página corresponde a un bucket y la búsqueda en estas se realiza como en las `slotted pages` de los archivos (con excepción de `compact` y `delete_all_permanently`, que son la única forma de eliminar de manera definitiva ya sea registros con `deleted=True` o todos los registros de la página).

El directorio es almacenado mediante una página de metadata de enteros (`MetadataPage`) donde cada entero $i_0, i_1, \dots, i_{2^{(D-10)}-1}$ representa las entradas del diccionario en grupos de 1024. Esto quiere decir que, con $D = 20$, `split` potencialmente podría acceder a 512 páginas solo para actualizar valores del directorio cuando se llegue a la profundidad máxima (cabe recalcar que, para modificar este comportamiento, solo habría que cambiar D a un valor menor). Sin embargo, para bases de datos con volúmenes de datos extraordinariamente grandes, el uso de un factor de profundidad $D = 20$ provocaría un incremento en la velocidad de la búsqueda al no tener que revisar muchos buckets de overflow sino solamente uno.

2.b.iv. Bitmap index scan

Es un índice no agrupado que asocia cada valor de una columna con los registros del heap que contienen dicho valor mediante una representación de bits. Su principal ventaja frente a otros índices consiste en que los conjuntos de registros pueden combinarse directamente mediante operaciones binarias (`AND` y `OR`, principalmente), sin acceder inicialmente al archivo de datos.

El planificador utiliza estas operaciones para combinar predicados: las condiciones conjuntivas (`AND`) se resuelven mediante intersecciones y las disyuntivas (`OR`) mediante uniones. Las búsquedas por rango admiten límites abiertos o cerrados y pueden rechazar rangos con demasiados valores distintos. El barrido recorre los `RIDs` ordenados por `page_id` y `slot_id`, permitiendo procesar conjuntamente los registros de una misma página del heap. Finalmente, la condición original se vuelve a evaluar sobre los registros recuperados y, si la máscara está vacía, no se accede al heap.

El índice se almacena como un archivo independiente con una cabecera, páginas de datos y páginas de overflow. Las páginas de datos utilizan una organización tipo slotted page, mientras que las máscaras que no caben completamente en una página continúan en cadenas de overflow. Al abrir el índice se construye en memoria un directorio que asocia cada valor con la página y slot de su entrada, permitiendo localizar directamente un bitmap. Las inserciones y eliminaciones actualizan los bits correspondientes, y las páginas liberadas se reutilizan mediante una lista de páginas disponibles.

El índice está diseñado para tablas organizadas como heap y permite búsquedas por igualdad y determinados rangos. Las condiciones de desigualdad no se representan directamente mediante complementos y el planificador no utiliza estadísticas de selectividad para decidir entre bitmap y un barrido secuencial. Por ello, cuando una condición coincide con una gran proporción de la tabla, el bitmap puede terminar accediendo a una cantidad de páginas similar a la de un recorrido completo.

2.c. Algoritmos externos

Los algoritmos externos permiten ejecutar operaciones cuyo volumen de datos puede superar la capacidad de la memoria principal. En lugar de mantener toda la relación en memoria, estos algoritmos dividen el procesamiento en fases y utilizan archivos temporales para almacenar resultados intermedios. De esta manera, la memoria utilizada depende principalmente de un presupuesto fijo y no del número total de registros.

2.c.i. External sorting (*k*-way merge)

El ordenamiento externo se emplea cuando la información que queremos ordenar no cabe en RAM.

Primero recibe pares formados por una clave y una fila serializada, y utiliza un presupuesto de mil elementos (definido mediante la constante `EXTERNAL_SORT_BUDGET` del ejecutor). Durante la primera fase, las entradas se acumulan hasta alcanzar dicho límite, se ordenan en memoria y se escriben como un run en un archivo temporal. Posteriormente, los runs se fusionan utilizando un min heap que mantiene como máximo una entrada de cada run. Por ello, durante la mezcla la memoria depende de la cantidad de runs y no de la cantidad total de registros.

Si todas las entradas caben dentro del presupuesto, el algoritmo evita la creación de archivos temporales y realiza directamente el ordenamiento en memoria. Esto evita el costo adicional de escritura y lectura de disco para consultas pequeñas.

La mezcla mantiene el orden de los registros con claves iguales mediante información adicional asociada a cada entrada. El heap considera tanto la posición de la entrada dentro de su run como el índice de la run, preservando así el orden original de registros con claves repetidas. Para el orden descendente se utiliza un comparador que invierte la relación entre claves durante la fase de mezcla.

Las claves se transforman a una representación binaria que permite compararlas lexicográficamente. Los enteros y números de punto flotante utilizan transformaciones específicas para conservar su orden numérico, mientras que los booleanos y cadenas uti-

lizan representaciones binarias correspondientes a sus tipos. Cada clave incluye además un indicador de tipo. Las filas, en cambio, se almacenan en los archivos temporales utilizando la misma representación serializada empleada por el archivo de datos.

2.c.ii. External hashing

En lugar de generar runs ordenadas, el external sorting distribuye los registros entre un número fijo de particiones (16 por defecto), utilizando el hash de la clave. Posteriormente, cada partición puede procesarse en memoria de forma independiente. En un hash join, se construye una tabla hash con las filas de una relación y se recorren las filas correspondientes de la otra relación para encontrar coincidencias. Como dos claves iguales siempre son asignadas a la misma partición, ninguna coincidencia requiere comparar registros de particiones diferentes.

La función de hash utilizada es FNV-1a de 64 bits aplicada sobre la representación codificada de la clave. Utilizar una función determinista permite que una misma clave sea asignada consistentemente a la misma partición. La codificación previa también distingue valores de tipos diferentes que pueden considerarse iguales en otros contextos del lenguaje, como `True` y `1`.

La tabla hash mantiene una lista de filas para cada clave, en lugar de asumir que cada clave aparece una sola vez. Esto permite representar correctamente joins entre relaciones con claves repetidas: si una clave aparece m veces en una relación y n veces en la otra, genera hasta $m \times n$ combinaciones. Para agregaciones, cada partición mantiene un acumulador por grupo. El ejecutor utiliza este mecanismo para implementar `COUNT`, `SUM`, `AVG`, `MIN` y `MAX`. Cuando no existe `GROUP BY`, todas las filas utilizan una misma clave de agrupación y terminan en una única partición.

Tanto el ordenamiento como el hashing utilizan generadores y eliminan sus archivos temporales mediante un bloque `finally`. Esto permite liberar los archivos incluso cuando el consumidor deja de solicitar resultados antes de completar el operador o cuando ocurre una excepción durante su ejecución. Si el directorio temporal fue creado por el propio operador, también se elimina al finalizar; si fue proporcionado externamente, únicamente se eliminan los archivos generados.

En el ejecutor, el ordenamiento externo se utiliza cuando no existe una estructura que permita obtener los resultados directamente en el orden requerido. Si la columna del `ORDER BY` posee un índice B+, el índice puede recorrerse en orden y evitar el ordenamiento externo. Del mismo modo, una consulta de vecinos más cercanos puede utilizar el R-tree para obtener los registros ordenados por distancia. Para resultados de agregaciones, el ejecutor puede ordenar en memoria la salida reducida de la fase de agrupación. El ordenamiento externo se utiliza principalmente para ordenar columnas sin un índice apropiado o resultados de operaciones como un join, cuyo esquema combinado no dispone de un índice propio.

La principal limitación de `ExternalHasher` es que el número de particiones está definido como una constante y no se ajusta dinámicamente al tamaño de la entrada o a la memoria disponible. Por ello, una partición individual podría superar la capacidad de

memoria para entradas suficientemente grandes. Tampoco existe particionado recursivo para resolver este caso ni un tratamiento específico del sesgo en la distribución de las claves. El algoritmo corresponde, por tanto, a una implementación de dos fases con un número fijo de particiones.

2.d. Algoritmos para consultas espaciales

2.d.i. k -NN (k nearest neighbors)

Devuelve los k puntos más cercanos a una ubicación dada, por ejemplo las diez gasolineras más próximas a una coordenada.

El ejecutor reconoce ese patrón mediante `_plan_knn`: un `ORDER BY` cuya expresión es una función de distancia con un operando literal, acompañado de un `LIMIT` y con un índice R-Tree sobre la columna involucrada. Cuando las tres condiciones se cumplen, la consulta se resuelve con el método `knn` del índice en lugar de ordenar la relación completa.

El algoritmo implementa una búsqueda **best-first** sobre una cola de prioridad (`heapq`). La cola almacena dos clases de elementos distinguidos por una etiqueta: nodos pendientes de expandir, priorizados por el `mindist` entre el punto de consulta y su MBR, y puntos ya alcanzados, priorizados por su distancia real. El recorrido procede extrayendo repetidamente el elemento de menor prioridad:

1. **Inicialización.** La raíz se encola con prioridad cero.
2. **Expansión de nodos internos.** Al extraerse un nodo interno, cada uno de sus hijos se encola con el `mindist` entre la consulta y el MBR correspondiente.
3. **Expansión de hojas.** Al extraerse una hoja, cada uno de sus puntos se encola con su distancia real al punto de consulta.
4. **Acumulación de resultados.** Al extraerse un elemento etiquetado como punto, este pasa directamente al conjunto de respuestas.
5. **Terminación.** El ciclo concluye cuando se han acumulado k puntos o cuando la cola queda vacía.

La corrección del criterio de parada descansa en una propiedad de la cola: como los elementos se extraen en orden creciente de prioridad, el primer punto que se extrae es necesariamente el más cercano de todos los que quedan por descubrir. Si existiera un punto más próximo, estaría bajo algún nodo cuyo `mindist` fuese menor, y ese nodo se habría extraído antes. En consecuencia, los puntos emergen de la cola ya ordenados por distancia y no hace falta una fase de ordenamiento posterior.

La implementación incorpora un contador incremental (`itertools.count`) como segundo componente de cada tupla encolada. Su función es romper los empates de prioridad: cuando dos elementos comparten la misma distancia, `heapq` procede a comparar el siguiente componente de la tupla, y sin ese contador intentaría comparar objetos `Point` o tuplas heterogéneas, lo que produciría un error en tiempo de ejecución. El contador garantiza además que los empates se resuelvan por orden de inserción.

El uso de una cola de prioridad reemplaza la alternativa de ordenar los hijos de cada nodo durante el descenso. Esa segunda estrategia obligaría a un ordenamiento por cada nivel visitado, mientras que la cola mantiene el orden global de forma incremental y permite además retomar ramas abandonadas si resultan más prometedoras que la actual.

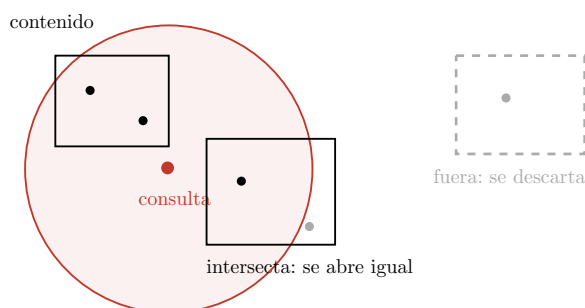
Cuando la consulta incluye además una cláusula **WHERE**, el índice puede devolver vecinos que el filtro posterior descarta, lo que dejaría el resultado por debajo de las k filas solicitadas. El ejecutor resuelve esto en `_filas_knn` solicitando un k mayor y duplicándolo sucesivamente hasta reunir las filas necesarias o hasta que el árbol deje de aportar candidatos nuevos. Los RID ya visitados se registran para no reevaluarlos en las iteraciones siguientes.

El árbol solo se emplea para orden ascendente. Un **ORDER BY ... DESC** solicita los puntos más lejanos, y la poda por `mindist` no permite descartar ramas en ese sentido: el rectángulo más cercano puede contener el punto más distante. En ese caso el ejecutor recurre al ordenamiento externo.

2.d.ii. Búsquedas por rango

Devuelve todos los puntos situados dentro de un radio respecto de una ubicación.

El método `radius_search` realiza un descenso recursivo más simple que el del k -NN, porque el umbral se conoce desde el inicio y permanece constante durante toda la búsqueda. En cada nodo interno se evalúa el `mindist` entre el punto de consulta y el MBR de cada hijo, y únicamente se desciende por aquellos cuyo valor no supera el radio. Al alcanzar una hoja se calcula la distancia real de cada punto y se conservan los que efectivamente quedan dentro.



Un rectángulo que interseca parcialmente la región de búsqueda se expande de todas formas, aunque algunos de los puntos que contiene queden fuera del radio: el MBR constituye una aproximación conservadora, y por eso la verificación exacta se difiere hasta las hojas. La relación es asimétrica y conviene precisarla: que un rectángulo interseque el radio no implica que contenga algún punto de la respuesta, pero que no lo interseque sí garantiza que no contiene ninguno. La poda, por tanto, nunca descarta resultados válidos, aunque puede visitar nodos que terminen sin aportar nada.

El ejecutor reconoce este caso en `_plan_espacial`, que examina la condición buscando una comparación cuyo lado izquierdo sea una función de distancia con un operando literal y cuyo operador sea `<` o `<=`. Los operadores `>` y `>=` no se aprovechan, dado que expresan la región exterior a un círculo y el `mindist` no ofrece un criterio de poda para ese complemento.

El R-Tree dispone también de `range_search`, que recibe un rectángulo en lugar de un radio y conserva los puntos contenidos en él. Su recorrido es análogo, sustituyendo la comparación de `mindist` por la intersección entre rectángulos (`Rectangle.intersects`). Esta variante se emplea internamente en la consulta de polígonos.

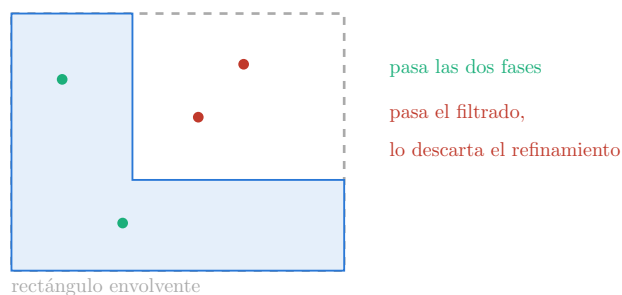
2.d.iii. Intersección con polígonos

Devuelve los puntos contenidos en un polígono arbitrario, por ejemplo las sucursales que se encuentran dentro de los límites de un distrito.

A diferencia de las dos consultas anteriores, un polígono no admite comparación directa contra los MBR del árbol: su contorno es irregular y los rectángulos no lo aproximan con la precisión suficiente para servir de criterio de poda. El método `polygon_search` lo resuelve mediante el esquema de dos fases habitual en sistemas de bases de datos espaciales, conocido como **filter and refine**.

Fase de filtrado. Se construye el rectángulo envolvente del polígono con `Rectangle.from_points` y se invoca `range_search` sobre él. Esta fase opera exclusivamente con comparaciones entre rectángulos, resulta económica y descarta la mayor parte del conjunto, pero produce un superconjunto de la respuesta: un punto puede situarse dentro del rectángulo envolvente y fuera del polígono.

Fase de refinamiento. Cada candidato se verifica con `point_in_polygon`, que implementa el algoritmo de **ray casting**. El procedimiento traza un rayo horizontal desde el punto y recorre los lados del polígono contando las intersecciones: para cada par de vértices consecutivos determina si el lado cruza la horizontal del punto, y en caso afirmativo calcula la abscisa del cruce e incrementa el contador si queda a la derecha. Un número impar de cruces indica que el punto se encuentra en el interior. El anillo se considera implícitamente cerrado, de manera que el segmento entre el último vértice y el primero se evalúa sin necesidad de repetir la coordenada inicial en la declaración.



La necesidad de la segunda fase se aprecia con polígonos cóncavos. En una figura en forma de L, una porción considerable del rectángulo envolvente queda fuera del

contorno, de modo que el filtrado deja pasar puntos que no pertenecen a la respuesta. Omitir el refinamiento produciría un resultado incorrecto, razón por la cual las pruebas de esta consulta contemplan específicamente ese caso.

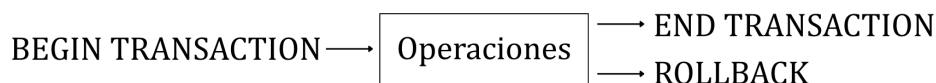
El costo de la consulta se descompone en consecuencia: la primera fase equivale a una búsqueda por rango sobre el rectángulo envolvente, y la segunda añade un recorrido lineal en el número de vértices por cada candidato. La eficiencia depende por tanto de dos factores: qué proporción del conjunto cubre el rectángulo envolvente, y qué tan ajustadamente el polígono llena ese rectángulo. Una figura alargada en diagonal constituye el peor caso, pues su envolvente abarca un área muy superior a la del propio polígono.

3. Transacciones, concurrencia y manejo de conflictos

La implementación de transacciones en esta base de datos combina un administrador de transacciones, un registro WAL (*Write-Ahead Log*), un administrador de locks y un módulo de recovery, los cuales proceden a explicarse en las secciones posteriores.

3.a. Transacciones

Una transacción agrupa varias operaciones y mantiene un estado único durante toda su ejecución. El ciclo normal es:



Al iniciar una transacción el `TransactionManager` asigna un identificador único y registra un evento en el WAL. Mientras la transacción está activa pueden ejecutarse operaciones, confirmar sus cambios con `END TRANSACTION` o cancelarlos con `ROLLBACK`. El `COMMIT` sigue un orden de durabilidad: primero se escribe el registro `COMMIT` en el WAL; solo después, la transacción se marca como confirmada y se liberan sus locks. De esta manera, el sistema no informa una confirmación antes de que el registro correspondiente sea durable.

Cada registro del WAL contiene, entre otros datos, el identificador de la transacción, el LSN, el `prev_lsn`, el archivo afectado, el tipo de recurso y las imágenes `before` y `after`. La cadena de `prev_lsn` permite recorrer las operaciones de una transacción en el orden inverso necesario para ejecutar UNDO.

3.b. Atomicidad y UNDO

La atomicidad busca evitar que una transacción deje solamente una parte de sus cambios. Si una sentencia produce un error dentro de una transacción explícita, el executor aborta la transacción, ejecuta el UNDO disponible y libera sus locks.

El WAL registra las operaciones CRUD con la información necesaria para deshacerlas:

- En un `INSERT`, el UNDO elimina la fila utilizando su clave primaria.
- En un `DELETE`, el UNDO reinserta los valores originales.

- En un **UPDATE**, el **UNDO** restaura la imagen anterior de la fila.
- Las operaciones se deshacen en orden inverso al que fueron registradas.
- Cada **UNDO** genera un registro **CLR (Compensation Log Record)**.

Cuando un **UPDATE** conserva el tamaño físico del registro, puede actualizarse el mismo slot y mantener el **RID** original. Si el nuevo valor cambia el tamaño de la fila y ya no cabe en el slot, la implementación puede borrar y volver a insertar el registro; en ese caso, la información lógica de la fila se conserva, pero el **RID** puede cambiar.

3.c. UNDO físico

Además del registro lógico de la operación, el buffer pool conserva una copia de la imagen original de cada página. Cuando una página se modifica dentro de una transacción, el **WAL** registra sus imágenes físicas **before** y **after** antes de que la página pueda escribirse en disco.

El **UNDO** físico restaura los recursos modificados por una transacción abortada: páginas de datos o de índices, headers de archivos, páginas asignadas durante la transacción, cambios de tamaño producidos por un truncado.

Al restaurar una página, el buffer manager actualiza tanto el archivo como el frame que pudiera estar cargado en memoria. Esto evita que una página restaurada vuelva a sobrescribirse al cerrar el motor.

Los registros físicos se distinguen de los registros **CRUD** mediante **resource_type**, por lo que no se intentan interpretar como payloads **JSON** de filas.

3.d. REDO y recuperación

Cuando el motor se abre, **RecoveryManager** identifica las transacciones confirmadas y las transacciones que quedaron activas en el **WAL**.

El proceso se realiza en dos etapas:

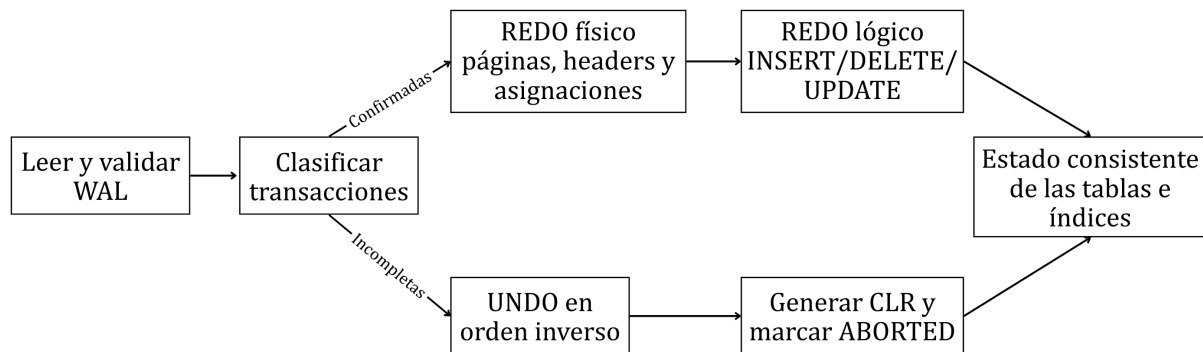
1. Se ejecuta redo sobre los cambios de transacciones confirmadas.
2. Se ejecuta **UNDO** sobre las transacciones que no tienen **COMMIT**.

El **REDO** lógico reaplica operaciones **INSERT**, **DELETE** y **UPDATE** de forma idempotente. Un **INSERT** no se duplica si la clave ya existe, un **DELETE** no falla si la fila ya fue eliminada y un **UPDATE** puede reconstruir el estado final esperado.

El **REDO** físico reaplica imágenes **before/after** de páginas, headers, asignaciones y truncados. Si una página contiene varios cambios consecutivos, las imágenes se procesan en cadena. Si la página ya se encuentra en el estado final, el registro no se vuelve a aplicar. Si contiene cambios posteriores que no pertenecen a la cadena, recovery evita sobrescribirlos.

El protocolo **WAL** también se aplica antes de escribir páginas modificadas por el buffer pool: el registro físico se fuerza antes de que la página dirty pueda llegar al archivo. El cierre normal escribe un **CHECKPOINT** durable después de cerrar las tablas y completar las operaciones pendientes.

FLUJO TRAS CIERRE INESPERADO O REAPERTURA DEL DBMS



3.e. Control de concurrencia

El control de concurrencia se realiza mediante un **LockManager** en memoria. Los locks se asocian a recursos, normalmente tablas, y a un identificador de transacción o de sesión.

Se soportan tres modos:

- **SHARED**: lock de lectura. Varias transacciones pueden mantenerlo simultáneamente.
- **UREAD**: lock de actualización. Es compatible con lectores **SHARED**, pero solo una transacción puede mantenerlo. Sirve para reservar la intención de actualizar y evitar que dos transacciones intenten promoverse simultáneamente.
- **EXCLUSIVE**: lock de escritura. Solo una transacción puede mantenerlo y excluye a lectores, locks **UREAD** y escritores de otras transacciones.

La compatibilidad principal es:

Lock existente	Solicitud SHARED	Solicitud UREAD	Solicitud EXCLUSIVE
Ninguno	Sí	Sí	Sí
SHARED de otra transacción	Sí	Sí	No
UREAD de otra transacción	Sí	No	No
EXCLUSIVE de otra transacción	No	No	No
Lock propio	Sí	Sí	Sí, si no hay otros lectores

Los locks son reentrantes por lo que una misma transacción puede adquirir nuevamente un recurso y cada adquisición se libera por separado. `release_all()` libera todos los recursos retenidos por una transacción al finalizarla.

Las lecturas adquieren **SHARED**. **INSERT** y **DELETE** adquieren **EXCLUSIVE**. **UPDATE** adquiere primero **UREAD** mientras identifica las filas que cumplen el **WHERE**; si encuentra filas para modificar, promueve el lock a **EXCLUSIVE** antes de ejecutar la mutación.

Una adquisición incompatible espera mediante una condición. Si se alcanza el timeout configurado, se lanza `LockTimeoutError`; el executor aborta la operación y libera los recursos adquiridos. Los escritores pendientes tienen prioridad sobre lectores nuevos, reduciendo el riesgo de inanición de escritores.

El sistema no implementa un detector formal de deadlocks. Para reducir el riesgo de esperas circulares, los locks de varias tablas se adquieren en orden lexicográfico y el timeout funciona como mecanismo de escape.

3.f. Integración con el executor

Cada sesión conserva como máximo una transacción explícita activa. Las sentencias principales tienen el siguiente efecto:

- `BEGIN TRANSACTION` crea la transacción y guarda su identificador en la sesión.
- `END TRANSACTION` escribe un `COMMIT` durable, libera los locks y limpia el estado de la sesión.
- `ROLLBACK` ejecuta `UNDO` lógico y físico, libera los locks y limpia el estado de la sesión.

En una transacción explícita, los locks se conservan hasta el `COMMIT` o el `ROLLBACK`. En autocommit, cada sentencia utiliza un identificador efímero de sesión y libera sus locks al terminar.

El executor registra también `CREATE TABLE` y `CREATE INDEX` cuando se ejecutan dentro de una transacción. Si la transacción se revierte, se eliminan la tabla o el índice creados. Si se confirma, recovery puede reconstruirlos cuando el registro WAL indica una transacción confirmada.

4. Frontend

El cliente web de la base de datos ha sido diseñado como una terminal de consultas interactiva (inspirada en herramientas como pgAdmin o DBeaver), con el objetivo de proveer una interfaz visual para la gestión de datos, el análisis de planes de ejecución y el monitoreo en tiempo real de las transacciones.

El frontend está desarrollado utilizando **Next.js** y **React**, aprovechando el renderizado del lado del cliente para mantener una experiencia de usuario fluida y reactiva. La interfaz se divide en dos modos de visualización principales: el entorno de consultas (**Queries**) y el laboratorio de transacciones (**Transactions**).

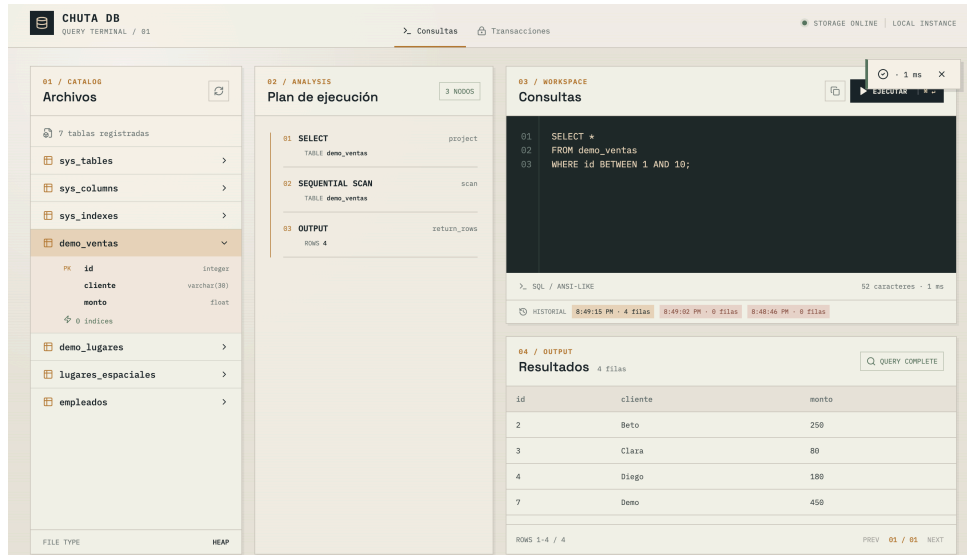
4.a. Estructura general y funcionamiento

El entorno de consultas se compone de cinco paneles interactivos:

1. **Catálogo (Catalog):** Consulta la API del motor para listar las tablas del sistema, mostrando sus esquemas, tipos de datos e índices disponibles.
2. **Análisis (Analysis):** Traduce el plan de ejecución retornado por el parser y el planificador del backend (ej. *Index Scan*, *External Sort*) en un árbol visual, mostrando filas procesadas.
3. **Área de trabajo (Workspace):** Un editor de texto SQL.

4. **Salida (Output):** Una tabla de datos paginada que renderiza las tuplas devueltas por el motor, manejando conversiones de tipos (como tuplas espaciales a texto) para su correcta visualización.
5. **Panel Espacial (Spatial):** Un lienzo de renderizado geográfico que reacciona a los datos en pantalla.

La comunicación con el motor (desarrollado en Python con FastAPI) se realiza mediante peticiones HTTP REST, manteniendo un identificador de sesión (X-Session-ID) para garantizar el correcto aislamiento de los bloqueos (locks) y transacciones por cliente.

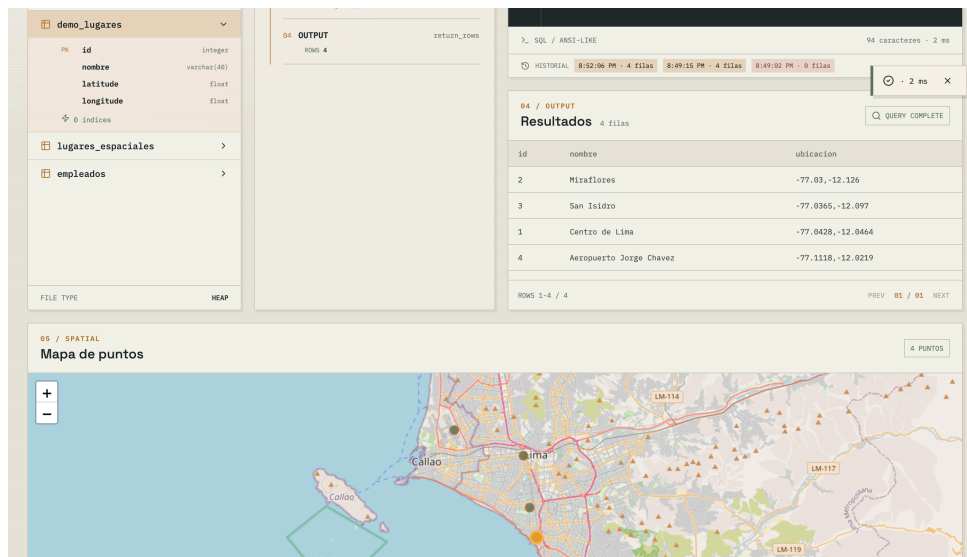


4.b. Integración con Leaflet

Para soportar las capacidades espaciales del motor (como la evaluación de distancias geodésicas y euclidianas), se integró la biblioteca **Leaflet** mediante un componente dinámico (MapPanel). A diferencia de un visor estático, el mapa está profundamente acoplado al ciclo de vida de las consultas SQL.

El flujo de integración funciona de la siguiente manera:

1. **Intercepción de resultados:** Cuando el usuario ejecuta una consulta (ej. `SELECT * FROM lugares WHERE distancia(...) < 5000`), el frontend analiza la metadata de las columnas retornadas.
2. **Detección de tipos:** El sistema busca automáticamente columnas definidas con el tipo nativo `POINT` (recibidas como arreglos `[longitud, latitud]`) o, en su defecto, columnas separadas de longitud y latitud.
3. **Renderizado dinámico:** Si se detecta geometría válida en el resultado activo, el mapa ignora la vista global de la tabla y proyecta exclusivamente las filas resultantes de la consulta.



5. Arquitectura final del DBMS

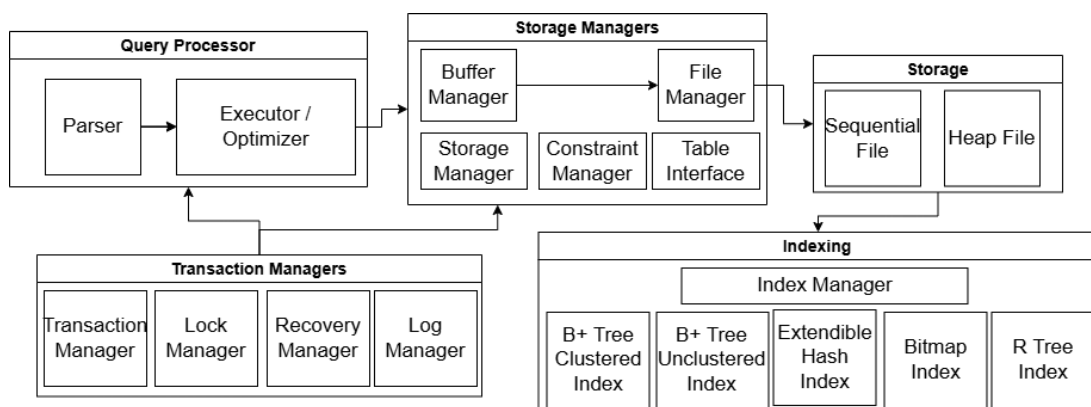


Figura 3: Diagrama de arquitectura del DBMS.

6. Benchmarking, comparaciones y testing

6.a. Heap file vs sequential file

Se midieron las dos organizaciones sobre los mismos datos (registros de dos enteros, claves insertadas en orden aleatorio) con tres tamaños. Cada número es el promedio de tres corridas.

Los resultados confirman lo esperado en inserción, búsqueda y reorganización, pero no en espacio: el archivo secuencial ocupa menos, no más.

6.a.i. Tiempo de inserción

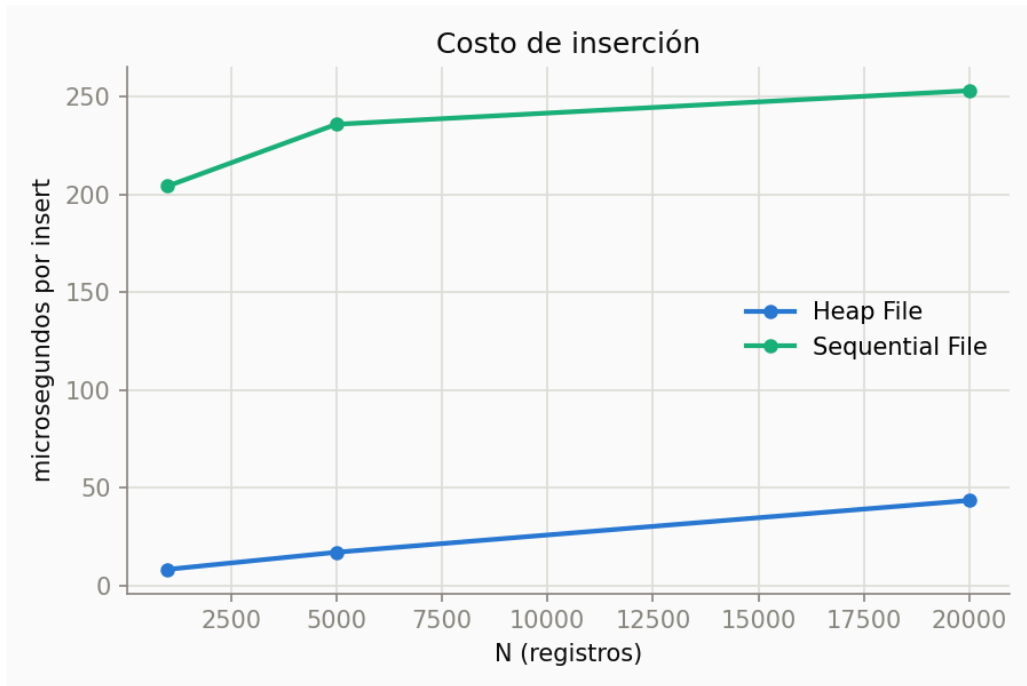


Figura 4: Costo de inserción por registro.

N	Heap (μs/insert)	Sequential (μs/insert)	Ventaja
1.000	8,2	204,1	24,9×
5.000	17,0	235,8	13,9×
20.000	43,5	252,9	5,8×

El heap inserta donde haya espacio libre, sin mantener ningún orden, y por eso gana en todos los tamaños. El secuencial paga unos 250 μ s por registro para ubicarlo en su posición y absorber la cadena de overflow; ese costo se mantiene estable al crecer N, porque el umbral de reorganización es proporcional al tamaño del archivo y no una cantidad fija.

La ventaja del heap se reduce de 25 $\times$ a 6 $\times$ porque su costo por inserción sube con N. Busca la primera página con espacio recorriendo el archivo desde el principio, de modo que a más páginas, más páginas llenas revisa antes de encontrar hueco. No es una propiedad de la organización sino de esta implementación: guardar la última página con espacio dejaría el costo constante.

6.a.ii. Tiempo de búsqueda por clave primaria

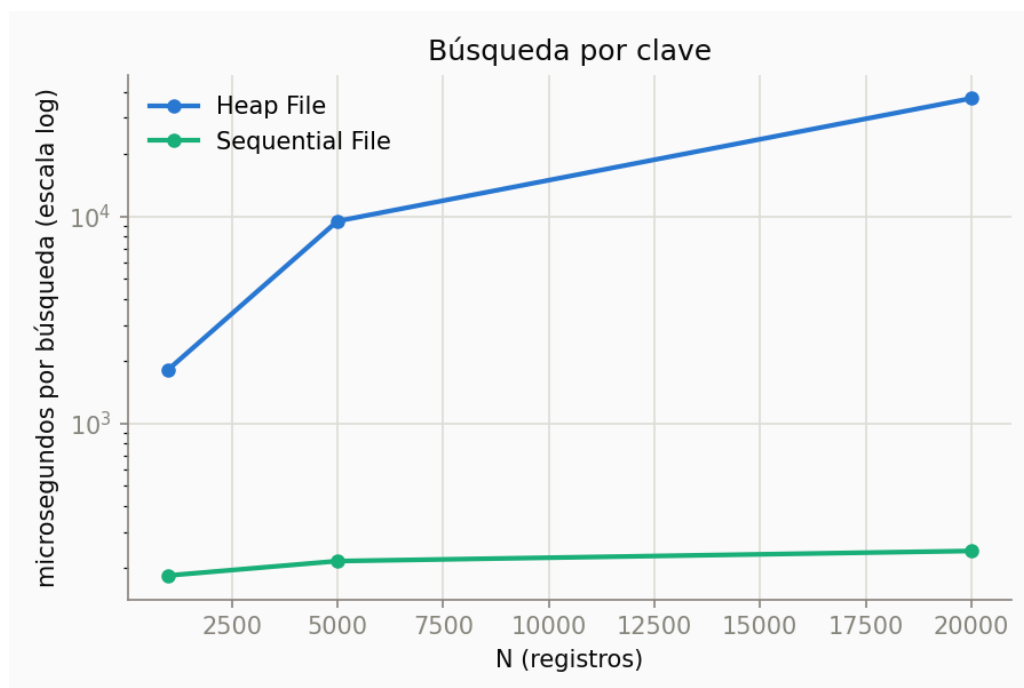


Figura 5: Búsqueda por clave. Escala logarítmica.

N	Heap (µs)	Sequential (µs)	Ventaja
1.000	1.825,6	184,8	9,9×
5.000	9.508,5	216,9	43,8×
20.000	37.267,9	242,8	153,5×

Acá se invierte la comparación, y la diferencia no es un factor constante sino creciente. El heap no indexa nada: para responder por clave tiene que recorrer el archivo completo, así que su tiempo crece proporcional a N (se multiplica por 20 cuando N se multiplica por 20). El secuencial mantiene los registros ordenados y ubica la clave con búsqueda binaria sobre las páginas, de modo que su tiempo se mantiene casi plano: entre el caso más chico y el más grande solo sube un 31%.

Con 20.000 registros el secuencial es 153 veces más rápido, y la brecha se abre con cada registro que se agrega.

Se midieron además otras dos operaciones de lectura, que ayudan a entender de dónde viene esa ventaja.

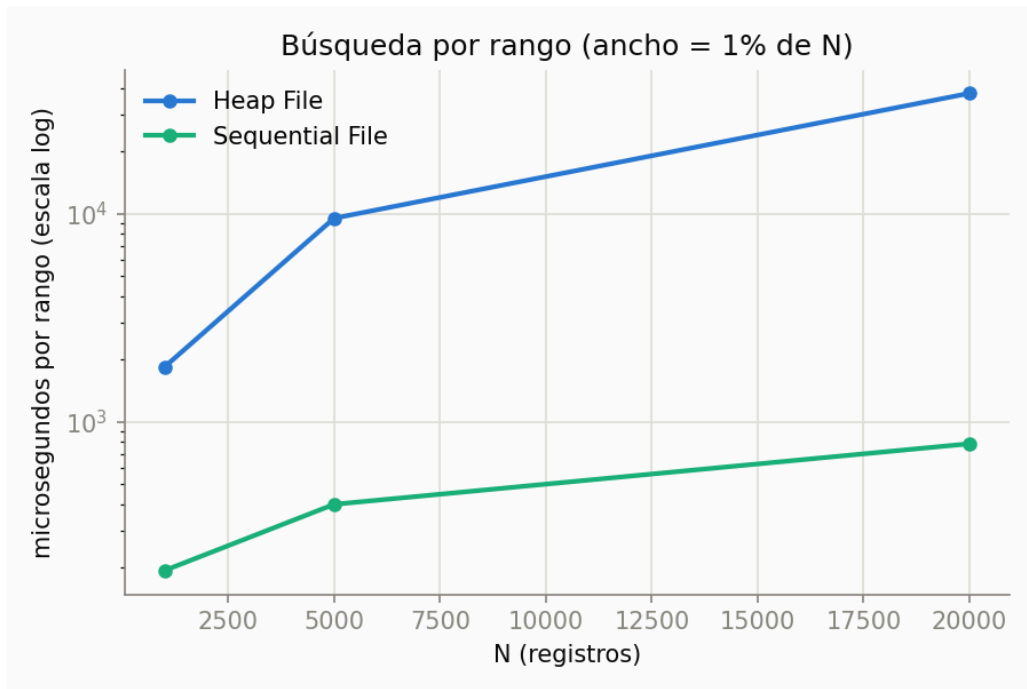


Figura 6: Búsqueda por rango, con un ancho del 1% de N. Escala logarítmica.

N	Heap (μs)	Sequential (μs)
1.000	1.833,6	192,3
5.000	9.510,3	400,1
20.000	37.941,6	782,5

Para el heap un rango cuesta lo mismo que una búsqueda exacta (37,9 ms contra 37,3 ms): como los registros no están ordenados, una clave del rango puede estar en la última página, así que hay que leer el archivo entero de todos modos. El secuencial ubica el inicio del rango y avanza hasta pasarse del límite superior. Su curva sí sube, pero no por el tamaño del archivo sino porque el ancho pedido es el 1% de N: con 20.000 registros devuelve 200 filas en lugar de 10.



Figura 7: Recorrido completo de la tabla en orden de clave.

N	Heap (ms)	Sequential (ms)
1.000	2,1	2,8
5.000	10,8	14,4
20.000	47,2	60,4

Este es el único caso de lectura donde gana el heap, y el resultado sorprende: el secuencial ya tiene los datos ordenados y el heap tiene que ordenarlos en memoria. La diferencia no está en la organización sino en cómo recorre cada uno. El heap carga una página y recorre sus 163 registros antes de pasar a la siguiente, es decir 124 lecturas de página para 20.000 registros. El secuencial sigue su lista enlazada y pide la página en cada registro: 20.000 lecturas. Las páginas están en el buffer y no hay E/S real, pero ese sobre costo supera lo que el heap paga por ordenar.

6.a.iii. Espacio en disco utilizado

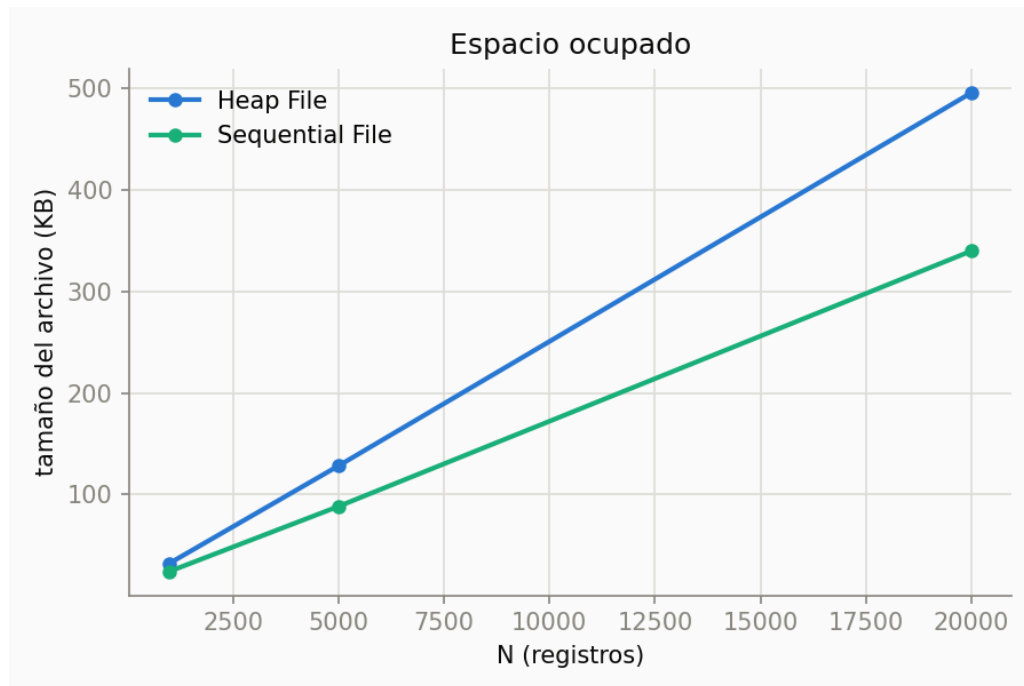


Figura 8: Tamaño del archivo de datos.

N	Heap (KB)	Sequential (KB)	Heap (B/reg)	Sequential (B/reg)
1.000	32,0	24,0	32,8	24,6
5.000	128,0	88,0	26,2	18,0
20.000	496,0	340,0	25,4	17,4

Este es el único resultado que contradice lo esperado: el archivo secuencial ocupa un 31% menos, no más.

La causa está en el formato de página. El heap guarda registros de longitud variable, así que cada uno necesita un **slot** de 8 bytes en el directorio de la página para saber dónde empieza y cuánto mide. El secuencial usa registros de longitud fija, donde la posición se calcula por aritmética y ese directorio no hace falta:

Componente	Heap	Sequential
Datos (dos enteros)	8	8
RID del siguiente	8	8
Marca de borrado	1	1
Slot del directorio	8	—
Total por registro	25 B	17 B

Con páginas de 4 KB entran 163 registros por página en el heap contra 240 en el secuencial. Para 20.000 registros eso predice 124 páginas (496 KB) y 84 páginas (344 KB) respectivamente, que es prácticamente lo medido. Los 25,4 y 17,4 bytes por registro de la tabla no son un efecto de medición: son el tamaño físico del registro en disco.

6.a.iv. Tiempo de reorganización

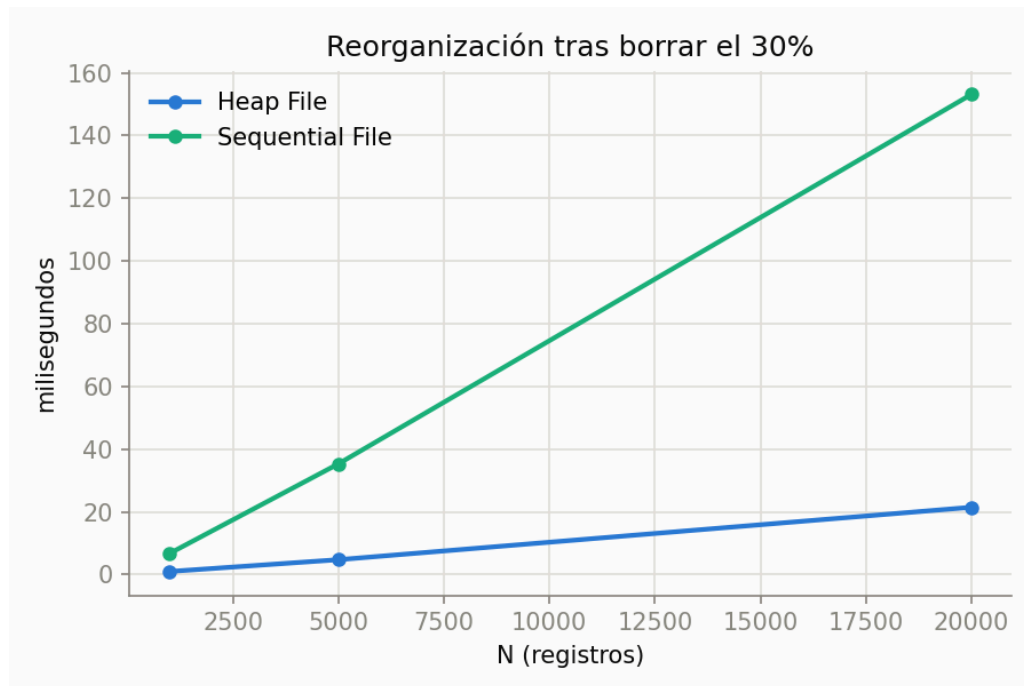


Figura 9: Reorganización después de eliminar el 30% de los registros.

N	Heap (ms)	Sequential (ms)	Ventaja
1.000	1,0	6,7	6,7×
5.000	4,7	35,3	7,5×
20.000	21,5	153,2	7,1×

El heap reorganiza unas siete veces más rápido, y la razón es que las dos operaciones no hacen lo mismo. El heap solo desfragmenta dentro de cada página: compacta los registros vivos y deja los slots libres para reutilizarlos en las inserciones siguientes, sin devolver páginas al archivo. El secuencial reescribe el archivo completo dejando únicamente los registros vivos, por eso cuesta más.

Eso también explica la diferencia de espacio después del borrado:

N = 20.000	Heap (KB)	Sequential (KB)
Archivo lleno	496,0	340,0
Tras borrar el 30%	496,0	240,0
Tras reorganizar	496,0	240,0

El secuencial se achica porque compacta; el heap mantiene su tamaño porque recicla el espacio internamente. Ninguna de las dos estrategias es incorrecta, recuperan el espacio en momentos distintos. En el secuencial la reorganización se dispara sola cuando el espacio muerto supera el 30%, por eso las dos últimas filas ya coinciden.

6.b. B+ clustered vs B+ unclustered vs extendible hashing

Se comparan cuatro técnicas de indexación sobre la misma clave (columna entera, registros $[k, k*2]$ con k barajado): **B+ clustered** sobre `SequentialFile`, **B+ unclustered** sobre `HeapFile`, **B+ unclustered (seq)** —un índice secundario sobre `SequentialFile` que indexa la propia clave de ordenamiento del archivo— y **Hash extensible**. Cada número es el promedio de 3 corridas para $N = 1000, 10000, 50000$.

Los resultados esperados de arriba se cumplen solo en parte: el hashing extensible **no** gana en búsqueda exacta a estos tamaños (lo explica la subsección correspondiente), y la variante seq resulta competitiva con el clustered en rango y orden pese a ser un índice no agrupado.

6.b.i. Búsqueda por igualdad exacta

Índice	N=1000 (μ s)	N=10000 (μ s)	N=50000 (μ s)	Tendencia
B+ clustered	22.3	28.9	31.3	log N, la más rápida
B+ unclustered	112.1	111.5	112.3	plana, indirectión al heap
B+ unclustered (seq)	79.6	82.1	97.2	entre ambas
Hash extensible	45.3	60.5	118.0	sube con N por el scan de bucket

El resultado contradice la expectativa de que el hash gane las búsquedas exactas. El hashing extensible tiene un costo **fijo** de 5 páginas por operación (verificado en `tests/indexes/test_hash_complexity.py`), que no es cero: el B+ con su fanout necesita solo 1–3 páginas hasta $N = 100000$, menos que esas 5. Por eso el **clustered** gana claramente en todos los N , y a $N = 50000$ el hash pasa a ser incluso el **más lento** de los cuatro (118.0 μ s). La variante **seq** se ubica entre el clustered y el unclustered-heap: y se degrada por la misma indirección de todo índice no agrupado (índice $\rightarrow$ fetch del dato) más un chequeo del contador de reorganización, pero le gana al unclustered-heap porque sus RID, al indexar la clave de ordenamiento del archivo, quedan casi en orden físico.

6.b.ii. Búsqueda por rango

Ancho de rango 1% de N ; eje en escala logarítmica por la diferencia de órdenes de magnitud del hash.

Índice	N=1000 (μ s)	N=10000 (μ s)	N=50000 (μ s)	Tendencia
B+ clustered	112.4	297.9	829.7	mejor de los B+
B+ unclustered	98.6	230.2	764.0	competitivo

B+ unclustered (seq)	101.3	255.7	782.5	cercano al clustered
Hash extensible	3266.1	33613.8	169140.1	scan completo del heap

Acá sí se cumple la expectativa estructural: el hash no tiene orden entre claves, así que responder un rango lo obliga a un **scan completo del heap**, órdenes de magnitud más caro (169 ms a $N = 50000$ frente a < 1 ms de los B+). Entre los B+, los tres quedan muy cerca entre sí porque todos recorren las hojas enlazadas en orden de clave; la variante **seq**, al indexar la clave de ordenamiento del `SequentialFile`, lee casi en orden físico y se pega al clustered.

6.b.iii. Ordenamiento

Recorrido completo en orden de clave (`range_search` sobre todo el dominio).

Índice	N=1000 (ms)	N=10000 (ms)	N=50000 (ms)	Tendencia
B+ clustered	1.62	19.58	95.58	lineal en N
B+ unclustered	1.39	16.14	107.22	indirección al heap
B+ unclustered (seq)	1.23	18.05	83.22	mejor a N grande
Hash extensible	3.62	38.47	218.58	scan + sort

Mismo razonamiento que el rango: el hash debe recorrer todo el heap y ordenar (218 ms a $N = 50000$), mientras los B+ recorren las hojas ya ordenadas. La expectativa de que el unclustered gane «por poco» al clustered se cumple a N chicos pero se invierte al crecer N : a $N = 50000$ el unclustered-heap es el más lento de los B+ (107 ms) por la indirección hacia páginas del heap sin localidad, mientras que la variante **seq** es la más rápida (83 ms) porque sus lecturas caen casi en orden físico.

Conclusión. El hashing extensible solo conviene cuando hay exclusivamente búsquedas por igualdad y el tamaño justifica su $O(1)$ (a estos N ni siquiera gana la búsqueda exacta). El clustered domina cuando la carga es de rango/orden sobre la clave primaria. La variante **seq** muestra que un índice no agrupado que apunta a la clave de ordenamiento del archivo hereda la localidad del clustered en rango/orden —a $N = 50000$ incluso gana el ordenamiento—, al precio de compartir su costo de construcción y churn (100 s a $N = 50000$), ya que cada reorganización del `SequentialFile` fuerza un reindexado completo del B+.

6.c. Búsqueda secuencial vs R-tree vs PostGIS

En esta sección se compara el índice R-tree implementado con: una búsqueda secuencial, que recorre todos los puntos sin ninguna estructura auxiliar, y el índice GiST de PostGIS, que representa una implementación de producción. La evaluación considera

dos tipos de consulta sobre puntos geográficos en dos dimensiones: las consultas por rango, entendidas como la búsqueda de todos los puntos ubicados a una distancia máxima de un centro, y las consultas de los k vecinos más cercanos. Para cada técnica se midió el tiempo de construcción del índice, el tiempo promedio de consulta y el espacio que ocupa el índice en disco.

El R-tree sigue el algoritmo de Guttman (1984). Cada nodo ocupa una página de 4096 bytes administrada por el *buffer manager* del proyecto, con un *buffer pool* de 50 marcos. Las hojas almacenan únicamente las coordenadas del punto y el RID del registro (24 bytes por entrada), por lo que admiten hasta 170 entradas; los nodos internos almacenan el MBR de cada hijo y su número de página (36 bytes por entrada), con un máximo de 113 hijos. La inserción desciende por el hijo cuyo MBR requiere la menor ampliación y, cuando un nodo se desborda, lo divide mediante el *split* cuadrático. La consulta por rango descarta todo subárbol cuya distancia mínima al centro (MINDIST) supera el radio y verifica la distancia exacta al llegar a las hojas. La búsqueda k -NN es de tipo *best-first*: una cola de prioridad ordenada por MINDIST expande siempre el nodo o el punto más cercano a la consulta y se detiene al reunir k puntos. En ambos casos la distancia se calcula con la fórmula de haversine sobre una esfera de 6371 km de radio. Con esta métrica, MINDIST es una aproximación, ya que proyecta el punto sobre el rectángulo como si el plano fuera euclidiano; sin embargo, a la escala de los experimentos no produjo ninguna diferencia en los resultados.

Antes de medir tiempos, la correctitud del índice se validó con pruebas automáticas: diez pruebas sobre la lógica del árbol con almacenamiento simulado en memoria (inserción, división de nodos, búsqueda por rectángulo, búsqueda por radio con métrica euclidiana y haversine, k -NN, intersección con polígonos, borrado con reinserción y persistencia) y dos pruebas de extremo a extremo sobre un heap file real. En todas ellas, el resultado del árbol se compara con el de una búsqueda por fuerza bruta sobre los mismos puntos. Además, el propio *benchmark* verifica que cada una de las 1800 consultas medidas devuelva exactamente el mismo conjunto que la búsqueda secuencial.

Las condiciones experimentales fueron las siguientes:

- **Datos.** Conjuntos de 1000, 10 000 y 100 000 puntos con distribución uniforme en una región de $0.5^\circ \times 0.5^\circ$, equivalente a unos 55 km $\times$ 55 km. Se utilizó la semilla 42, de modo que las tres técnicas trabajan sobre los mismos puntos y los mismos centros de consulta.
- **Consultas.** Radios de 1, 5 y 10 km para las consultas por rango, y valores de k iguales a 10, 50 y 100 para k -NN. Cada tiempo reportado es el promedio de 100 consultas con centros aleatorios.
- **Búsqueda secuencial.** Recorre la lista completa de puntos en Python. En una consulta por rango calcula la distancia de cada punto al centro; en una consulta k -NN ordena todos los puntos por distancia.
- **R-tree.** Se ejecuta en el mismo proceso de Python. Los registros se guardan en un almacenamiento simulado en memoria para aislar el costo del índice del costo del archivo de datos.

- **PostGIS.** PostgreSQL 18 con PostGIS 3.6 en la misma máquina, con una tabla de tipo `geography(Point, 4326)` indexada con GiST. Las consultas por rango usan `ST_DWithin` y las de k -NN usan el operador de distancia `<->` con `LIMIT k`; todas se envían desde Python mediante `psycopg2`.
- **Construcción y espacio.** Se midieron una sola vez por cada tamaño del conjunto.

N	Construcción R-tree (s)	Construcción PostGIS (s)	Índice R-tree (KB)	Índice GiST (KB)	Altura R-tree
1000	1.28	0.07	36	72	1
10 000	14.56	0.41	340	632	1
100 000	197.53	4.77	3436	7496	2

Tabla 1: Tiempo de construcción y espacio en disco del índice según el volumen de los datos. *Elaboración propia.*

La construcción es el aspecto en que la diferencia con PostGIS es mayor: GiST construye el índice entre 19 y 41 veces más rápido, y la brecha crece con N . El R-tree inserta los puntos uno por uno; cada inserción desciende desde la raíz, escribe las páginas modificadas a través del *buffer pool* y, cuando un nodo se llena, ejecuta el *split* cuadrático, cuyo costo crece con el cuadrado del número de entradas del nodo. PostGIS, en cambio, inserta primero todas las filas en lote y luego construye el índice completo con código compilado en C. Al pasar de 10 000 a 100 000 puntos, el tiempo del R-tree aumenta 13.6 veces, algo más que el crecimiento de N . Esto es coherente con el costo logarítmico de cada inserción, puesto que el árbol gana un nivel, y con que, a ese tamaño, las 859 páginas del índice ya no caben en los 50 marcos del *buffer pool*, por lo que parte de las inserciones debe leer y escribir páginas en disco.

La búsqueda secuencial no tiene costo de construcción ni ocupa espacio adicional, puesto que solo necesita la lista de puntos. Esa es su principal ventaja frente a los índices, a cambio de recorrer el conjunto completo en cada consulta.

En espacio, el índice del R-tree ocupa entre el 46 % y el 54 % de lo que ocupa el índice GiST. El R-tree utiliza unos 35 bytes por punto, frente a valores de entre 65 y 77 bytes en GiST. Como cada entrada de hoja ocupa 24 bytes, la ocupación media de las hojas es cercana al 68 %, un valor esperable en un árbol cuyo *split* garantiza que cada nodo quede lleno al menos hasta la mitad. GiST, además de las coordenadas, almacena las cabeceras de página y de tupla propias de PostgreSQL. El uso de memoria principal no se midió directamente; en el caso del R-tree, este se encuentra acotado por el *buffer pool*, que retiene como máximo 50 páginas (200 KB) sin importar el valor de N .

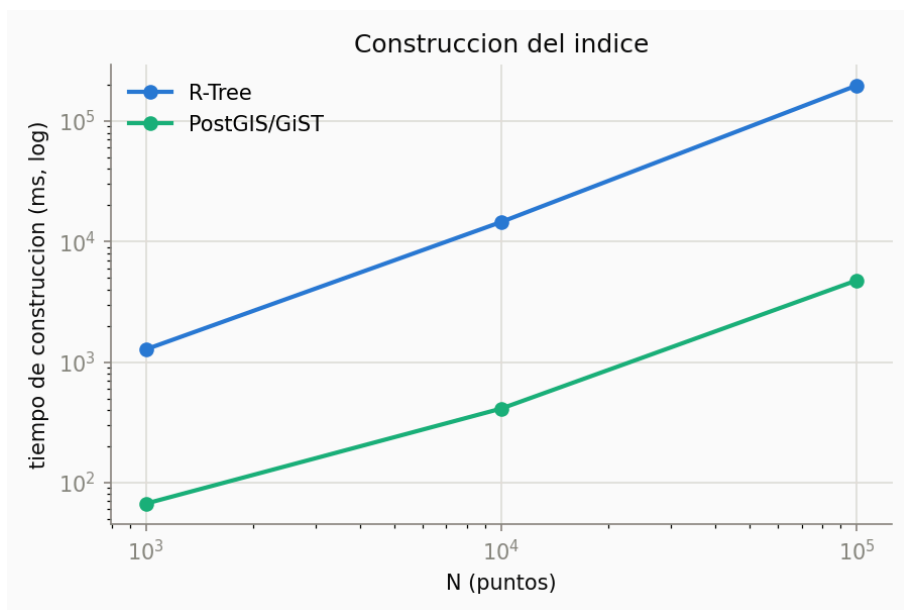


Figura 10: Tiempo de construcción del índice según N , en milisegundos y escala logarítmica. *Elaboración propia.*

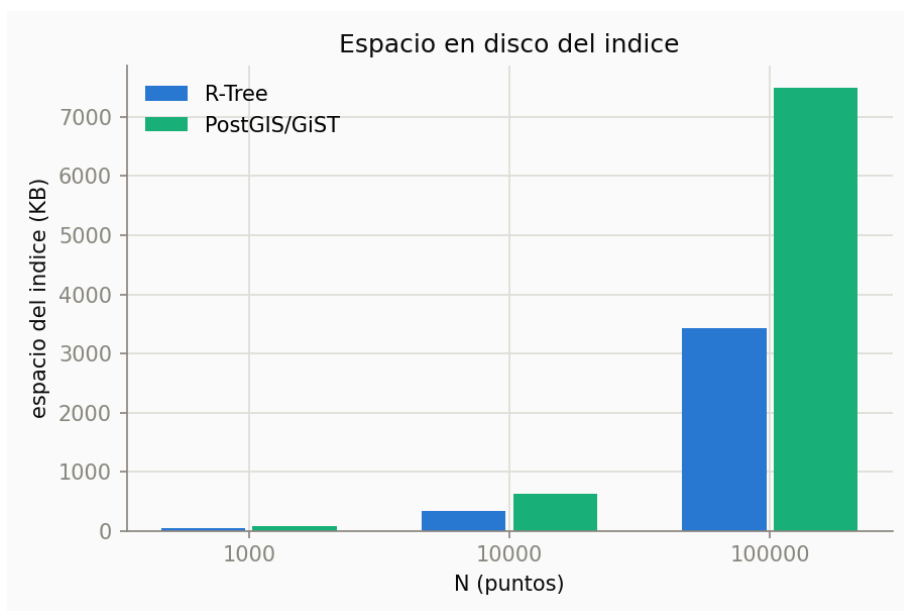


Figura 11: Espacio en disco del índice según N . *Elaboración propia.*

6.c.i. Consultas por rango

N	Radio (km)	Resultado (%)	R-tree (ms)	Secuencial (ms)	PostGIS (ms)	Aceleración
1000	1	0.09	1.16	2.23	1.01	1.93×
1000	5	2.31	1.42	2.13	0.69	1.50×
1000	10	8.75	1.71	1.84	1.05	1.08×
10 000	1	0.10	2.34	21.38	2.10	9.13×
10 000	5	2.42	5.10	19.21	3.55	3.77×
10 000	10	8.68	10.50	20.33	6.53	1.94×
100 000	1	0.10	4.95	173.56	2.62	35.04×
100 000	5	2.34	31.74	178.60	12.80	5.63×
100 000	10	8.56	99.12	196.08	34.70	1.98×

Tabla 2: Tiempo promedio de las consultas por rango. La última columna indica cuántas veces es más rápido el R-tree que la búsqueda secuencial. *Elaboración propia.*

La búsqueda secuencial presenta el comportamiento esperado de un recorrido completo, de costo $\mathcal{O}(N)$: su tiempo se multiplica aproximadamente por diez cada vez que N crece diez veces (de unos 2 ms a unos 20 ms, y luego a valores de entre 174 y 196 ms) y casi no depende del radio, porque calcula la distancia de todos los puntos sin importar cuántos formen parte del resultado.

El R-tree supera a la búsqueda secuencial en los nueve casos, y su ventaja crece con N . Con un radio de 1 km, la aceleración pasa de 1.9 veces con 1000 puntos a 9.1 veces con 10 000 y a 35 veces con 100 000. La causa es la poda: con un radio pequeño, MINDIST descarta casi todos los subárboles y la consulta visita solo las pocas hojas cercanas al centro, de modo que su costo depende de la altura del árbol y del tamaño del resultado, y no del total de puntos.

La ventaja disminuye, en cambio, a medida que crece el radio. Con 100 000 puntos pasa de 35 veces con 1 km a 5.6 veces con 5 km y a 2.0 veces con 10 km. Con un radio de 10 km, la consulta devuelve en promedio el 8.6 % del conjunto, unos 8560 puntos, y el círculo intersecta muchas más hojas; el trabajo se concentra entonces en recorrer y verificar puntos que sí pertenecen al resultado, trabajo que ningún índice puede evitar. Con 1000 puntos y un radio de 10 km la diferencia es de apenas un 8 %: el índice completo tiene solo 9 páginas y la lista que recorre la búsqueda secuencial es corta, por lo que ambos métodos realizan una cantidad de trabajo similar.

PostGIS es la técnica más rápida en los nueve casos, entre 1.1 y 2.9 veces más rápida que el R-tree. La diferencia es mínima con un radio de 1 km (1.1 veces con 1000 y con 10 000 puntos), porque ambos índices visitan pocas páginas y, en PostGIS, el costo fijo de la comunicación entre cliente y servidor representa una parte importante del tiempo total. Con 10 000 y 100 000 puntos la diferencia crece con el radio (con 100 000 puntos, de 1.9 a 2.9 veces), lo que es coherente con un menor costo por punto devuelto en el

código compilado de PostGIS que en la implementación en Python, donde cada entrada leída se convierte en un objeto.

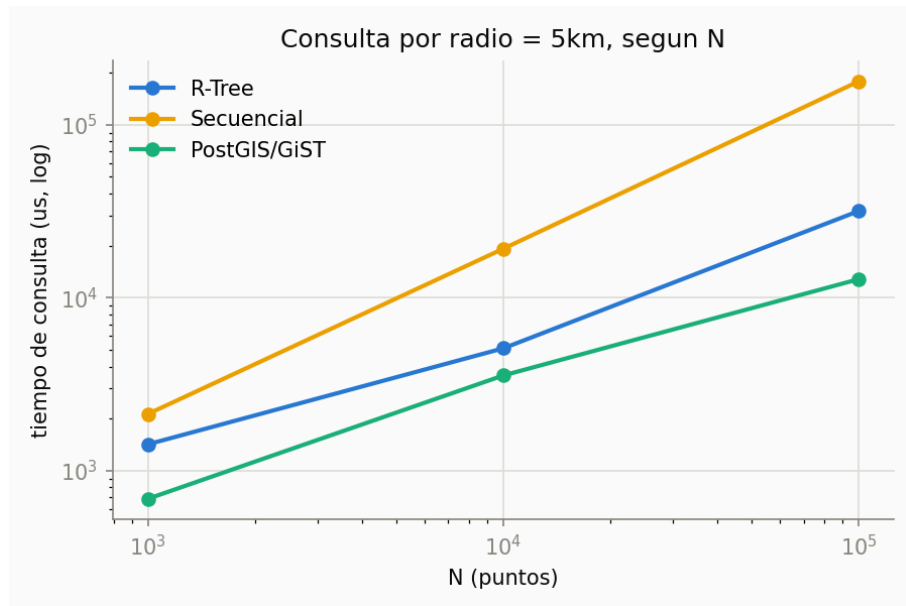


Figura 12: Tiempo de la consulta por rango con un radio de 5 km según N , en microsegundos y escala logarítmica. *Elaboración propia.*

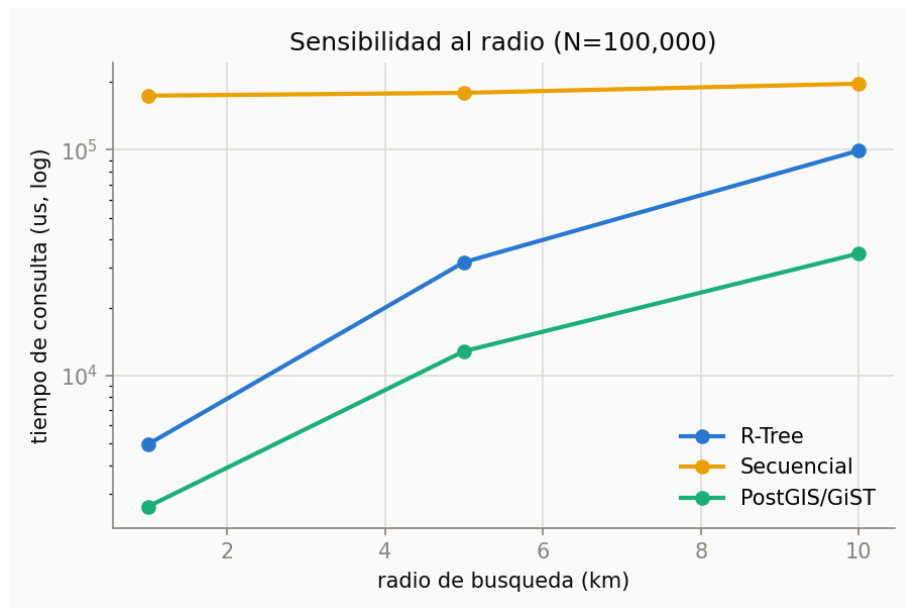


Figura 13: Tiempo de la consulta por rango según el radio, con 100 000 puntos, en microsegundos y escala logarítmica. *Elaboración propia.*

6.c.ii. Consultas de k -NN

N	k	R-tree (ms)	Secuencial (ms)	PostGIS (ms)	Aceleración
1000	10	1.58	2.61	0.54	1.65×
1000	50	1.77	2.11	0.82	1.19×
1000	100	2.34	2.09	1.17	0.89×
10 000	10	2.45	22.96	0.62	9.37×
10 000	50	3.33	20.84	1.02	6.25×
10 000	100	5.08	24.29	1.38	4.78×
100 000	10	4.28	253.84	0.87	59.24×
100 000	50	5.70	226.15	1.15	39.66×
100 000	100	7.64	219.91	1.56	28.77×

Tabla 3: Tiempo promedio de las consultas k -NN. La última columna indica cuántas veces es más rápido el R-tree que la búsqueda secuencial. *Elaboración propia.*

La búsqueda secuencial vuelve a crecer en proporción a N (de unos 2 ms a unos 22 ms, y luego a valores de entre 220 y 254 ms) y es prácticamente independiente de k , ya que en cada consulta calcula y ordena las distancias de todos los puntos, con un costo de $\mathcal{O}(N \log N)$.

El R-tree, en cambio, crece muy lentamente con N : con $k = 50$, su tiempo pasa de 1.77 ms a 3.33 ms y a 5.70 ms, mientras que N crece cien veces. La búsqueda *best-first* explica este comportamiento, puesto que la cola de prioridad expande primero los nodos con menor MINDIST y termina en cuanto reúne k puntos, de modo que nunca visita los nodos cuyo MBR está más lejos que el k -ésimo vecino. Por ello, la aceleración frente a la búsqueda secuencial alcanza 59 veces con 100 000 puntos y $k = 10$. El tiempo del R-tree sí aumenta con k , porque un valor mayor obliga a expandir más hojas antes de completar el resultado.

El único caso en que el R-tree resulta más lento que la búsqueda secuencial corresponde a 1000 puntos con $k = 100$ (2.34 ms frente a 2.09 ms). A ese tamaño, el índice tiene solo 9 páginas (una raíz y ocho hojas de unos 125 puntos cada una) y cabe completo en el *buffer pool*, de modo que ambos métodos trabajan enteramente en memoria principal y el acceso a disco no los diferencia. Además, $k = 100$ equivale al 10 % del conjunto, por lo que la búsqueda debe expandir varias hojas. Por cada punto de una hoja visitada, el R-tree convierte la entrada en un objeto, calcula su distancia y la inserta en la cola de prioridad, mientras que la búsqueda secuencial solo calcula la distancia y ordena. Con tan pocos puntos, este costo adicional por entrada iguala y supera el ahorro que aporta la poda. A partir de 10 000 puntos, el R-tree vuelve a ser claramente superior, con aceleraciones de entre 4.8 y 59 veces.

PostGIS es la técnica más rápida en los nueve casos, y su ventaja sobre el R-tree es mayor que en las consultas por rango: entre 2.0 y 5.0 veces, y cercana a 5 veces con 100 000 puntos. Su tiempo casi no varía con N (con $k = 10$, de 0.54 ms a 0.87 ms), ya que el operador $<->$ recorre el índice GiST en orden de distancia y entrega los vecinos

directamente. Una parte de ese tiempo corresponde al costo fijo de cada consulta entre cliente y servidor, que actúa como límite inferior.

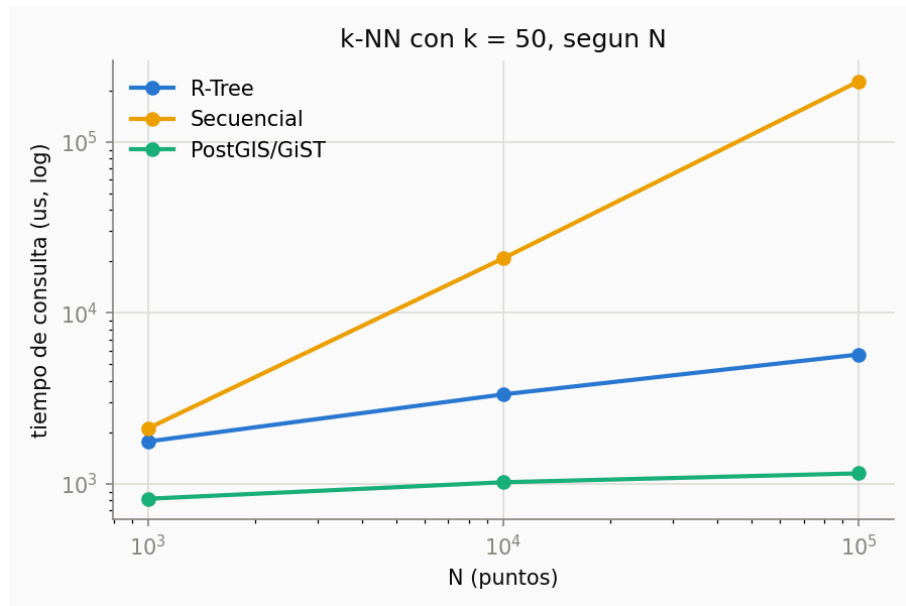


Figura 14: Tiempo de la consulta k -NN con $k=50$ según N , en microsegundos y escala logarítmica. *Elaboración propia.*

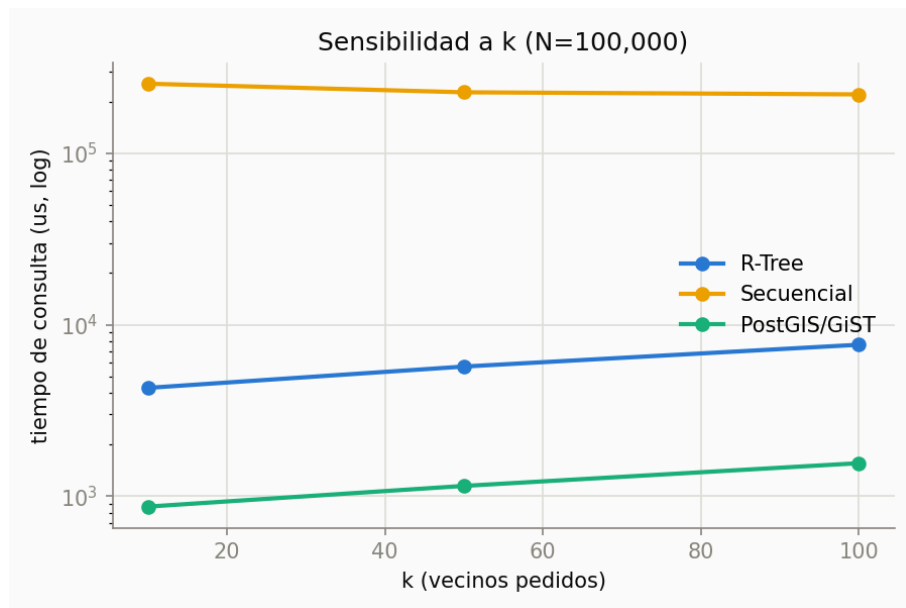


Figura 15: Tiempo de la consulta k -NN según k , con 100 000 puntos, en microsegundos y escala logarítmica. *Elaboración propia.*

6.c.iii. Discusión y limitaciones

En síntesis, el R-tree cumple el objetivo de superar a la búsqueda secuencial: lo hace en los nueve casos de consultas por rango y en ocho de los nueve casos de k -NN, con ventajas que crecen con el tamaño del conjunto y que alcanzan 35 y 59 veces, respectivamente, con 100 000 puntos. PostGIS supera a ambas técnicas en todas las consultas y construye el índice mucho más rápido, un resultado esperable en un motor

escrito en C y optimizado durante años; a cambio, su índice ocupa aproximadamente el doble de espacio en disco.

Los resultados deben interpretarse considerando las siguientes limitaciones:

- Los tiempos de PostGIS incluyen la comunicación con el servidor a través de un *socket* local, mientras que el R-tree y la búsqueda secuencial se ejecutan dentro del mismo proceso.
- El R-tree se midió con un almacenamiento simulado en memoria para los registros, mientras que PostGIS almacena además la fila completa en su tabla (72 KB, 672 KB y 6672 KB para los tres tamaños).
- PostGIS mide las distancias sobre un elipsoide y el R-tree, sobre una esfera. Por ello, el porcentaje de puntos devueltos difiere ligeramente entre ambas técnicas, en menos de 0.04 puntos porcentuales.
- El uso de memoria principal no se midió; se reporta, en su lugar, el espacio en disco del índice.
- La construcción se midió una sola vez por cada tamaño, sin repeticiones.